



UE21EC343BC3 DEEP LEARNING

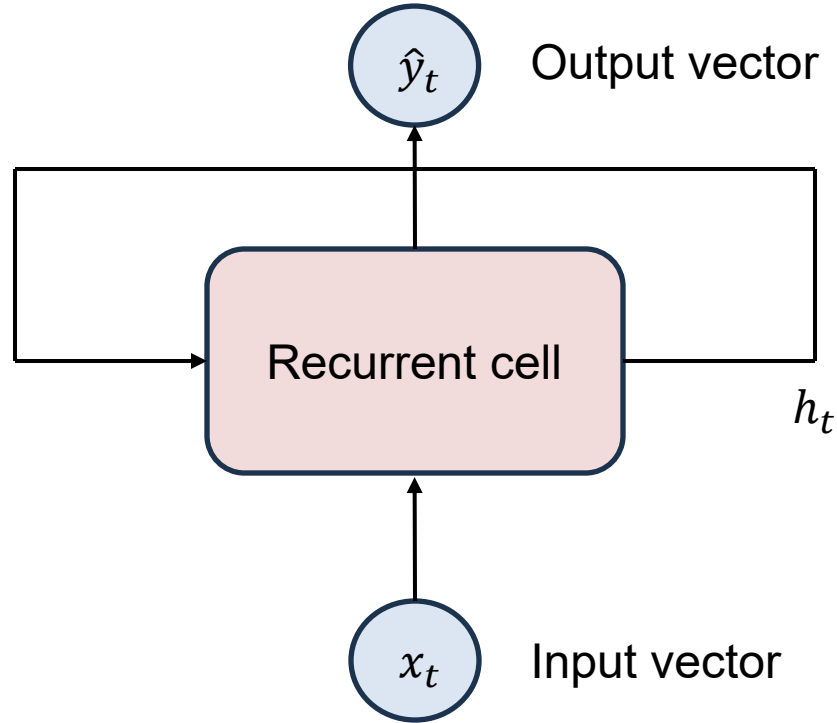
Dr. Rashmi N Ugarakhod

Department of Electronics and Communication Engineering

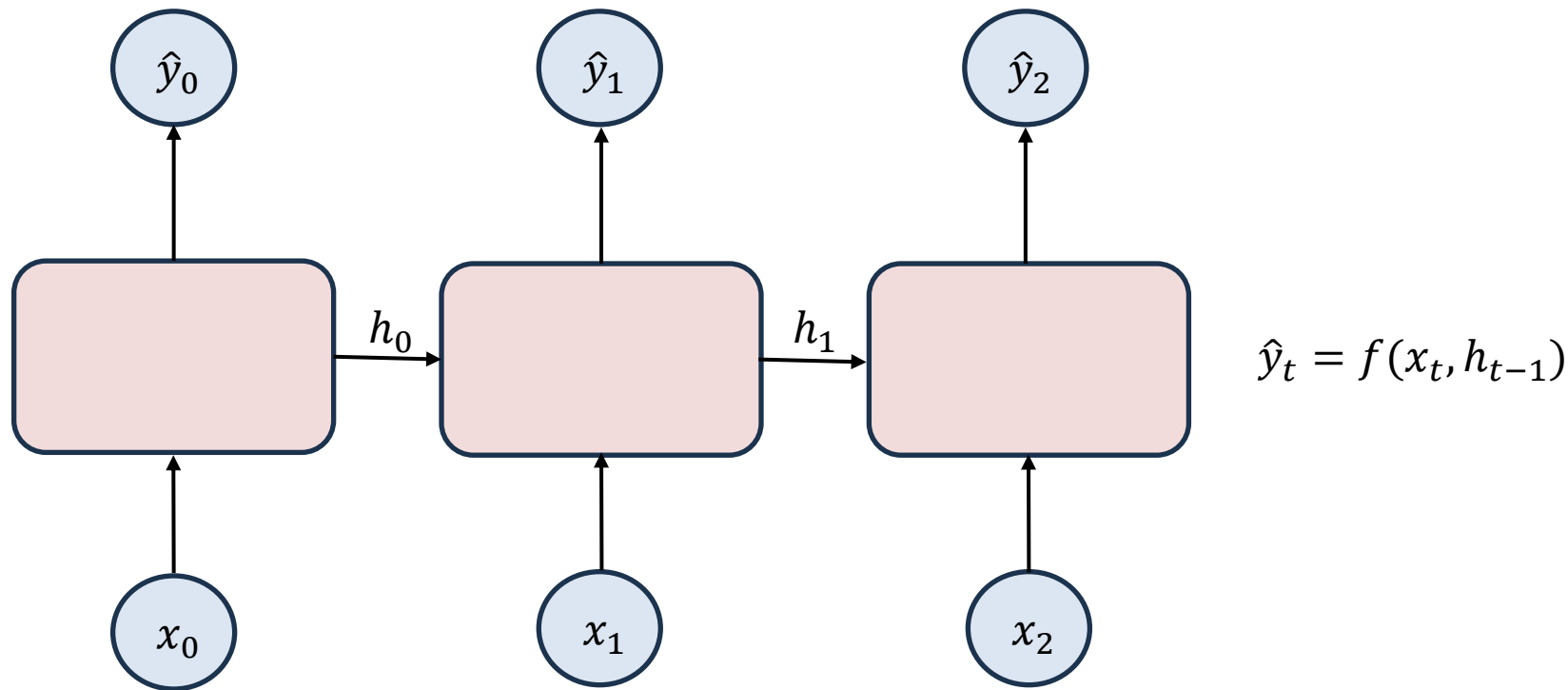
rashmin@pes.edu

Recurrent Neural Networks

Neurons with Recurrence

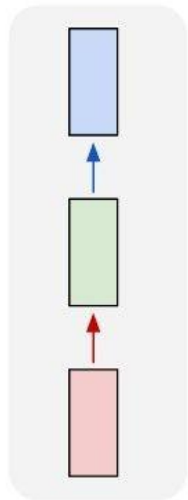


Neurons with Recurrence



“Vanilla” Neural Network

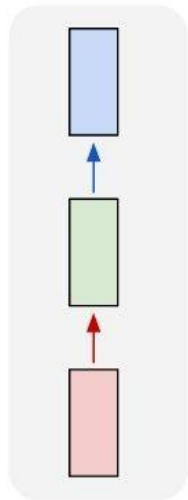
one to one



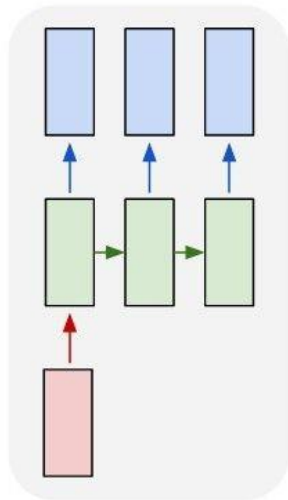
Vanilla Neural Networks

Recurrent Neural Networks: Process Sequences

one to one



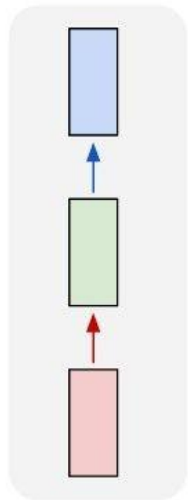
one to many



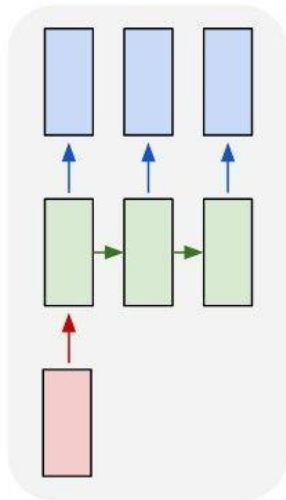
↖ e.g. **Image Captioning**
image -> sequence of words

Recurrent Neural Networks: Process Sequences

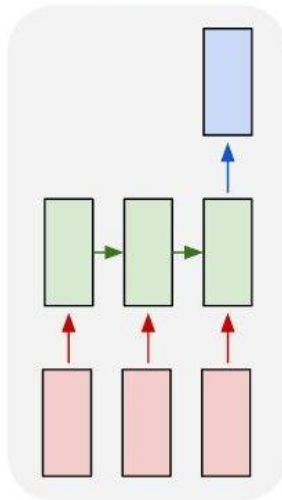
one to one



one to many



many to one

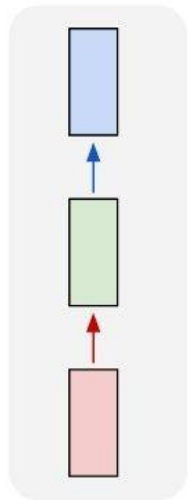


e.g. **action prediction**

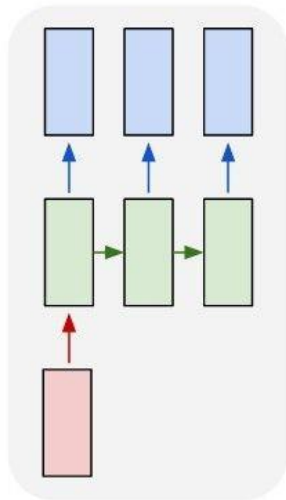
sequence of video frames -> action class

Recurrent Neural Networks: Process Sequences

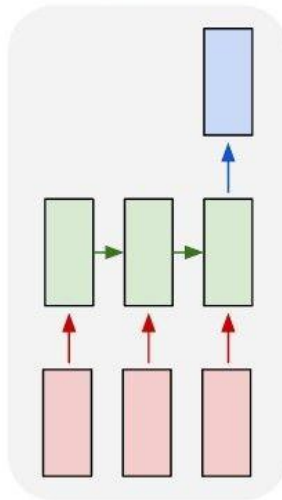
one to one



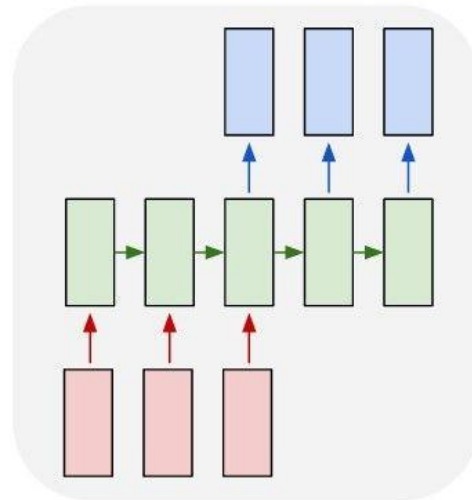
one to many



many to one



many to many

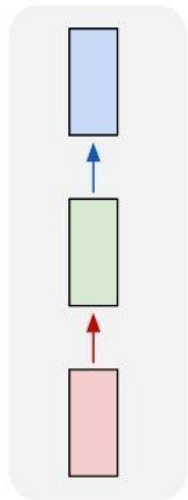


↖
E.g. Video Captioning

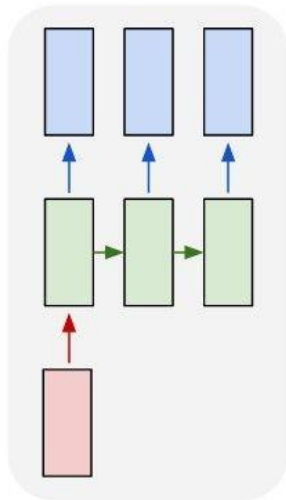
Sequence of video frames -> caption

Recurrent Neural Networks: Process Sequences

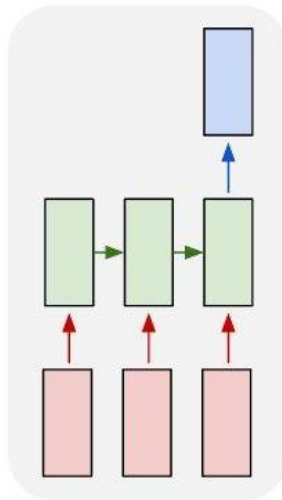
one to one



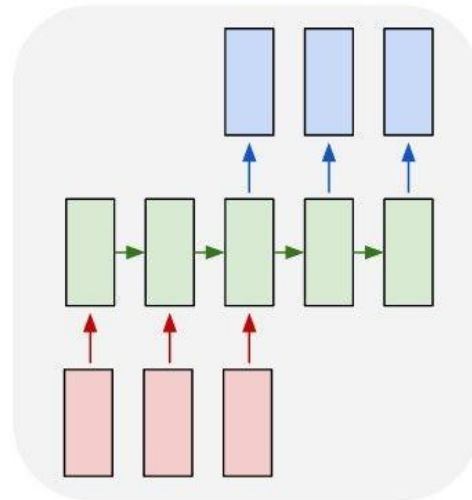
one to many



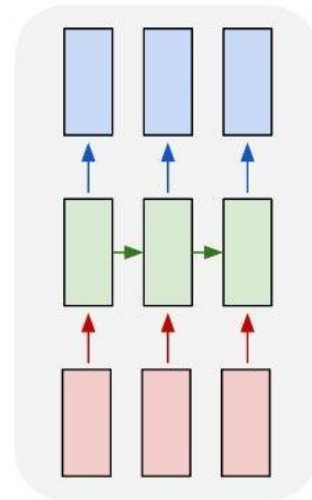
many to one



many to many



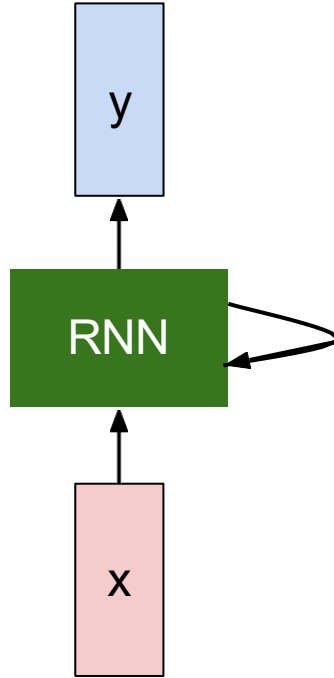
many to many



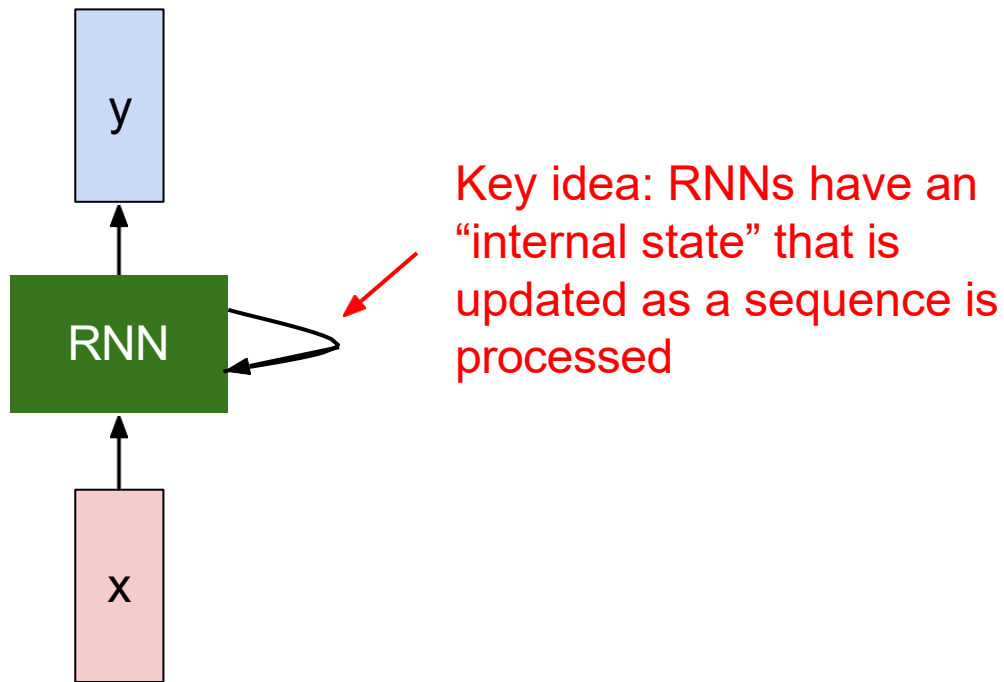
e.g. **Video classification on frame level**



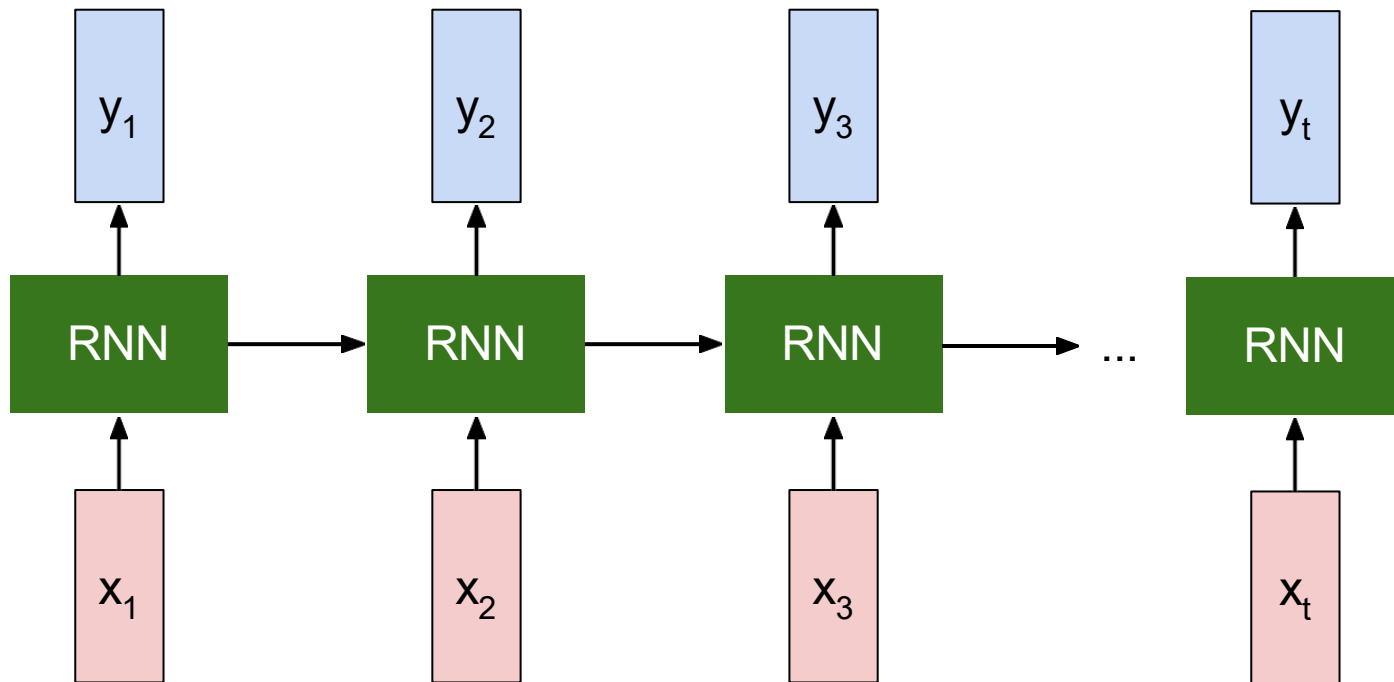
Recurrent Neural Network



Recurrent Neural Network



Unrolled RNN



RNN hidden state update

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

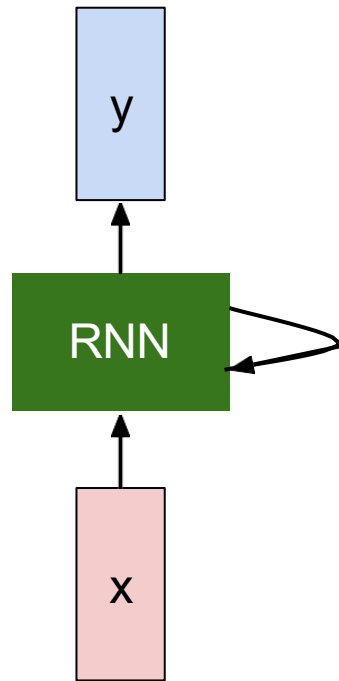
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with parameters W

old state

input vector at some time step



RNN output generation

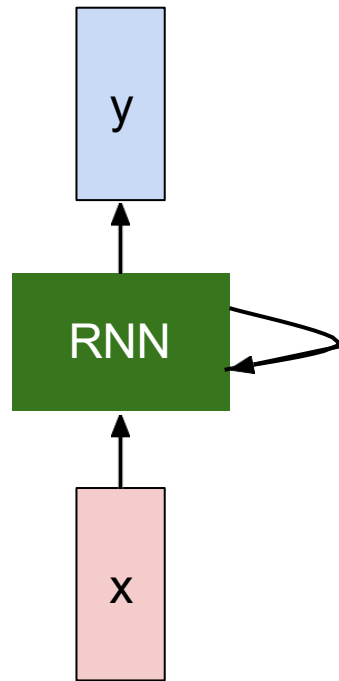
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{y_t} = \boxed{f_{W_{hy}}}(\boxed{h_t})$$

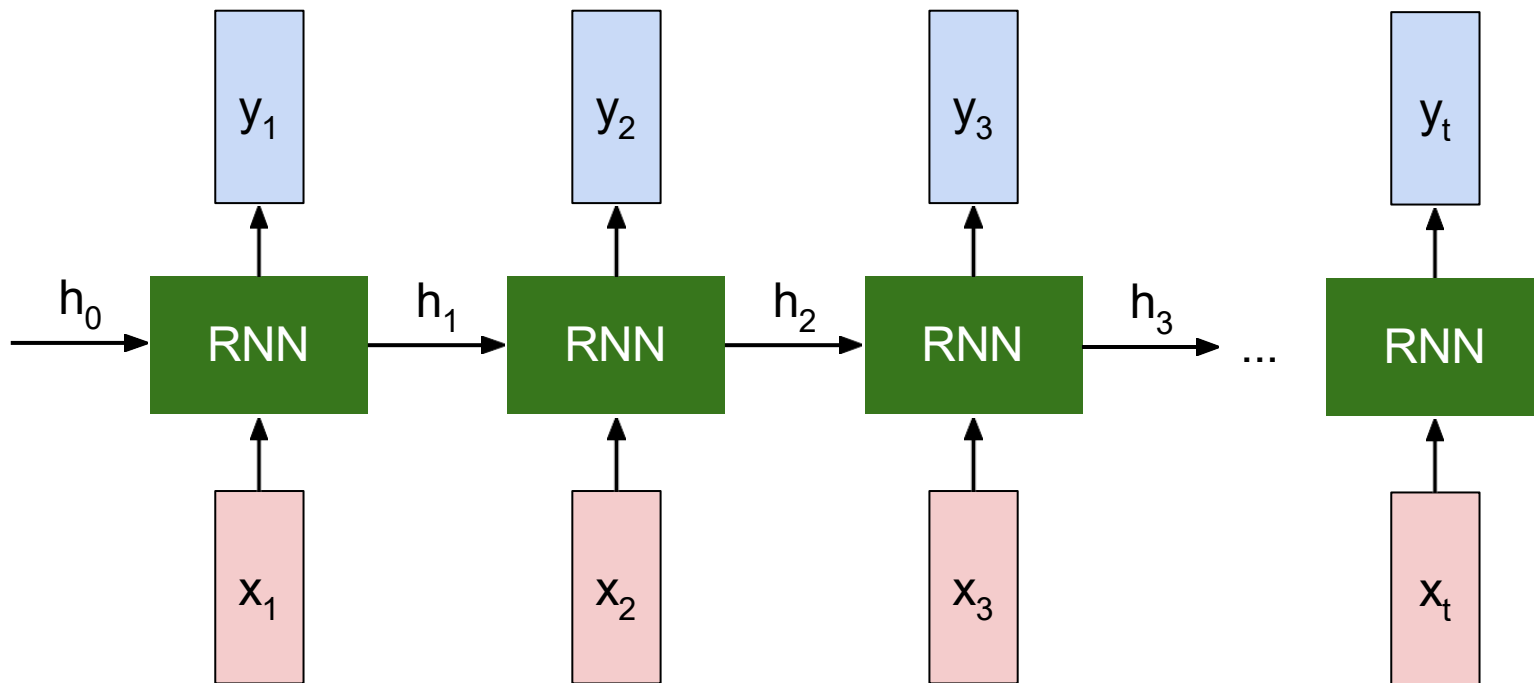
output

another function with parameters W_o

new state



Recurrent Neural Network

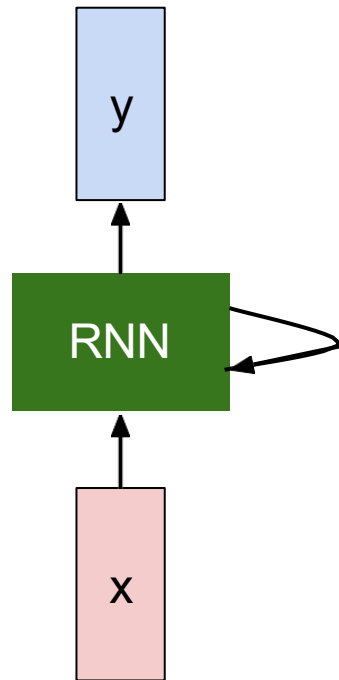


Recurrent Neural Network

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

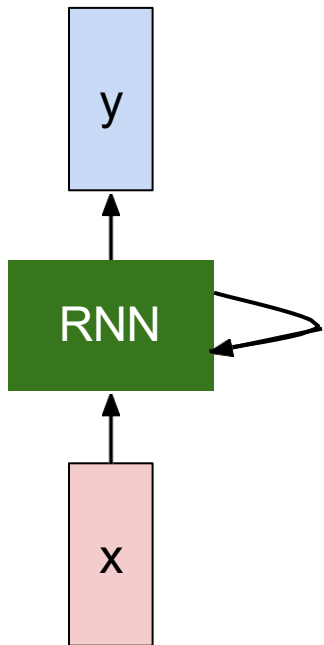
$$h_t = f_W(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.



(Vanilla) Recurrent Neural Network

The state consists of a single “*hidden*” vector h :



$$h_t = f_W(h_{t-1}, x_t)$$

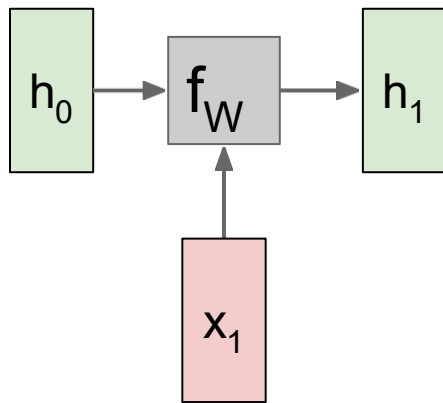


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

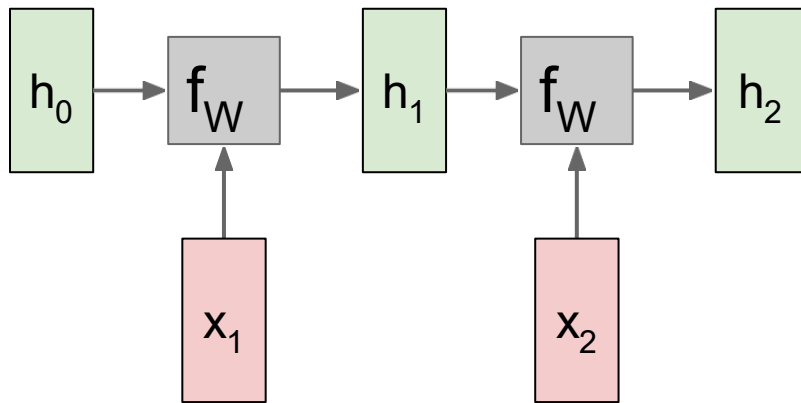
$$y_t = W_{hy}h_t$$

Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman

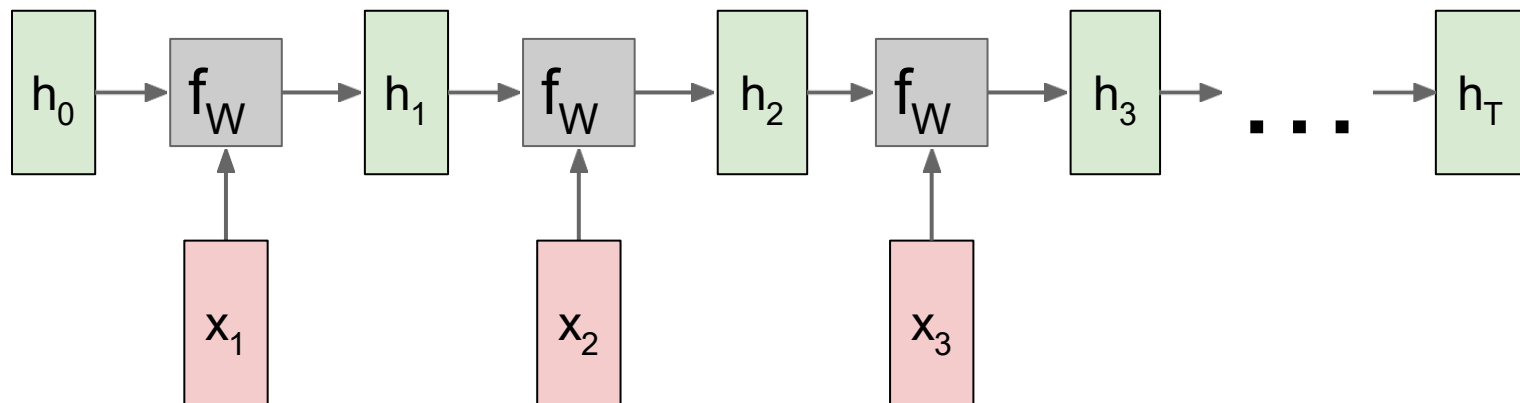
RNN: Computational Graph



RNN: Computational Graph

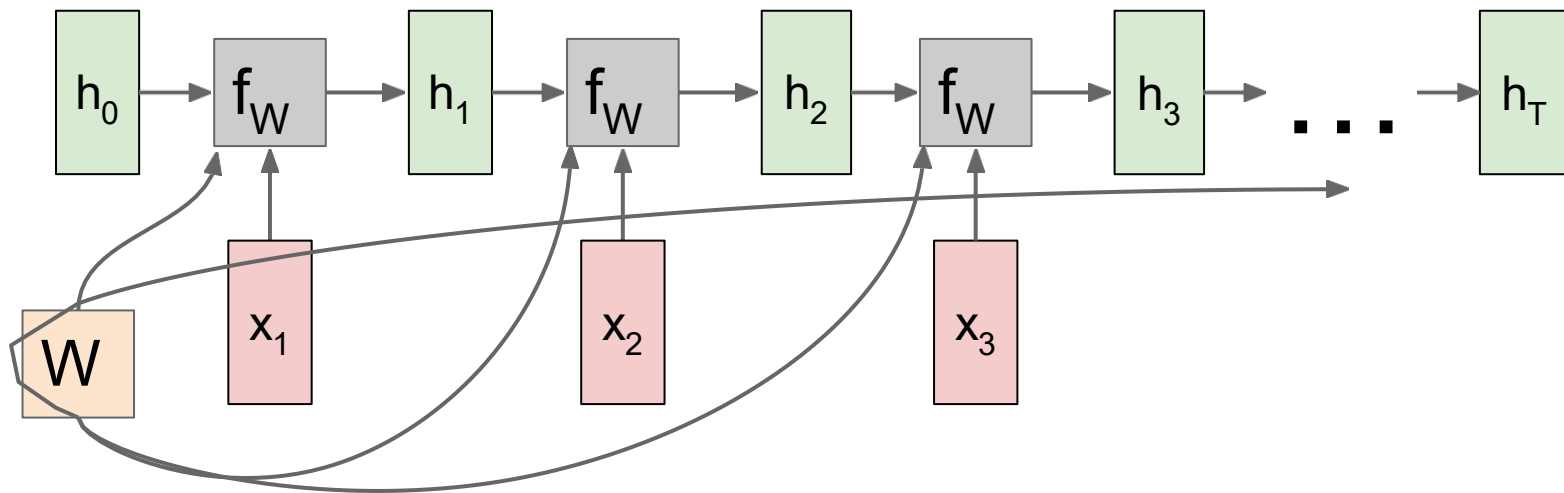


RNN: Computational Graph

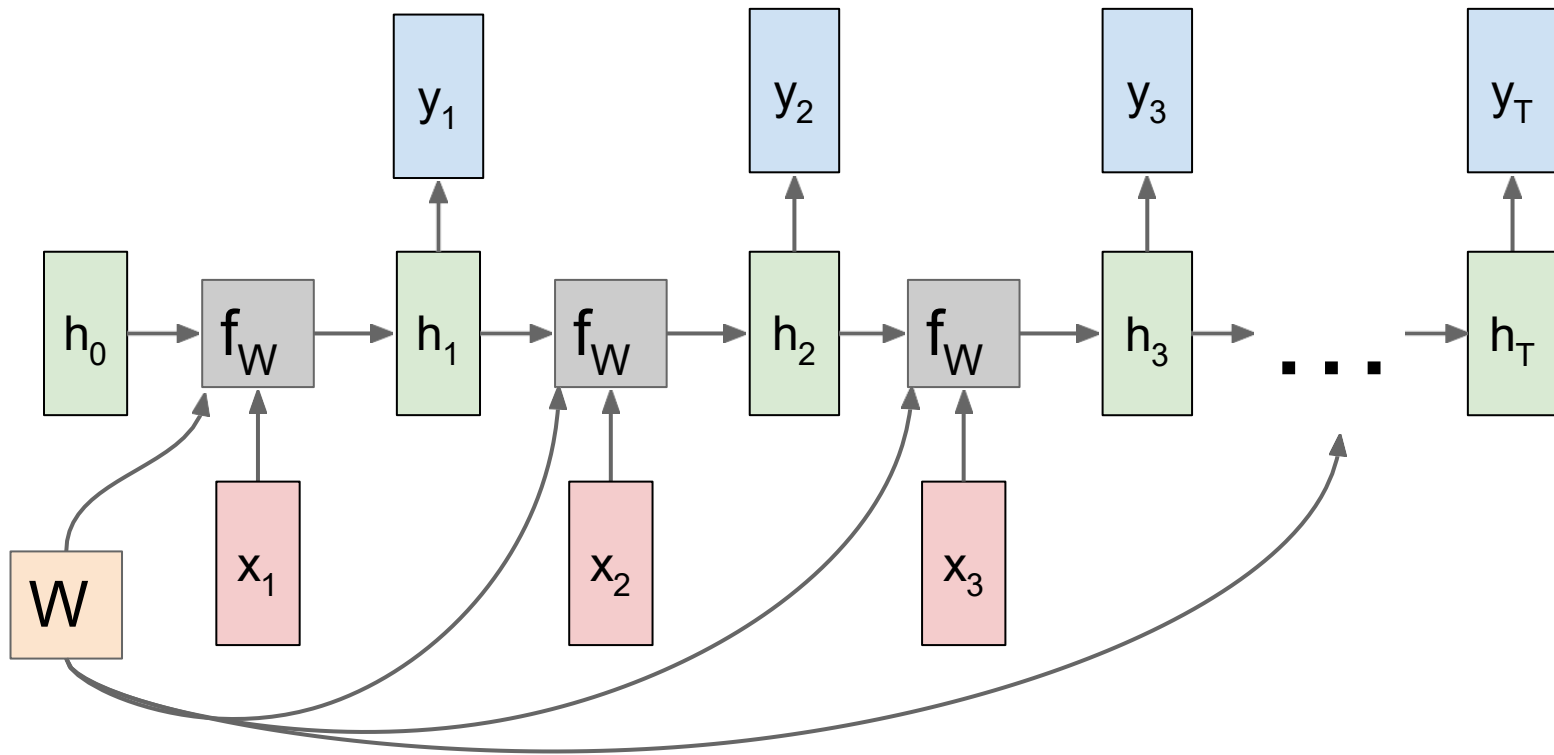


RNN: Computational Graph

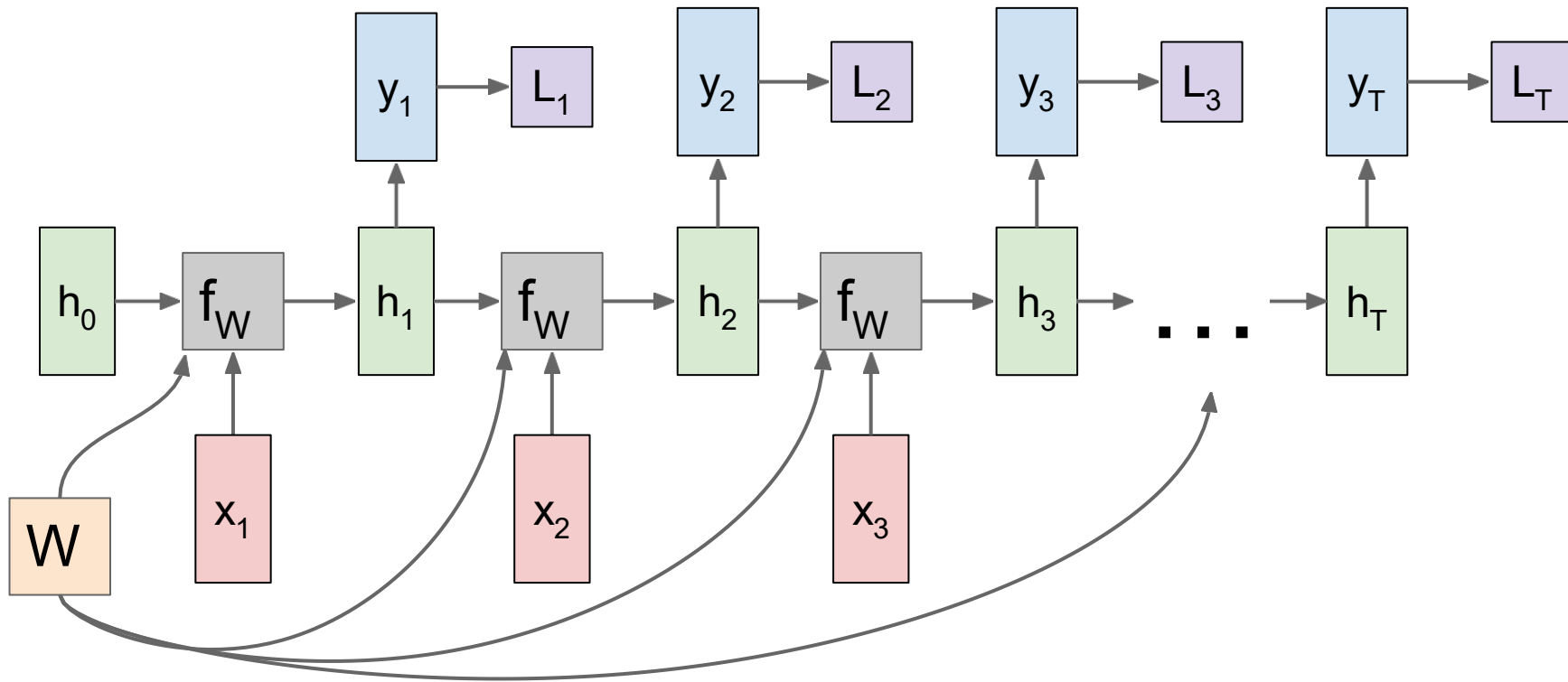
Re-use the same weight matrix at every time-step



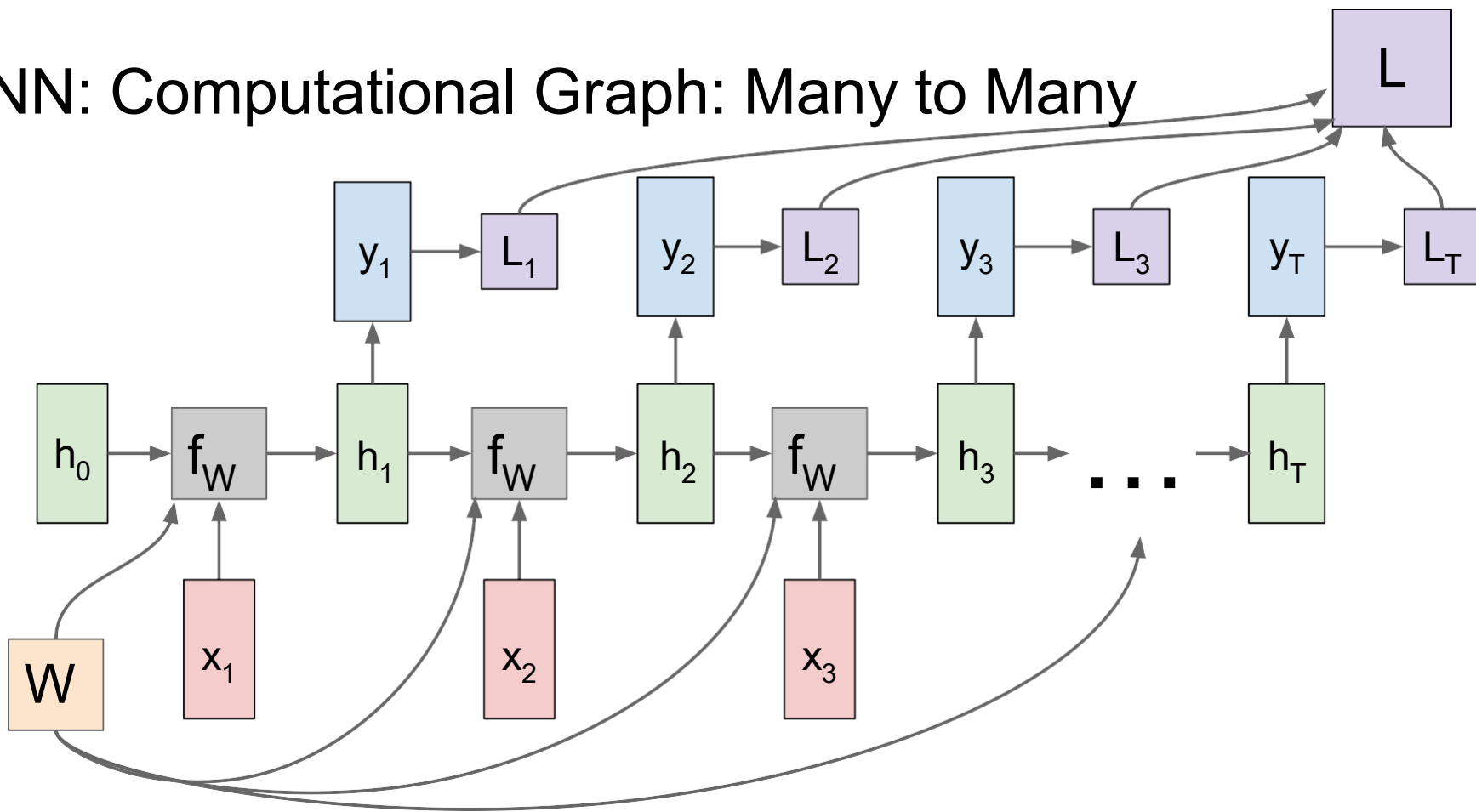
RNN: Computational Graph: Many to Many



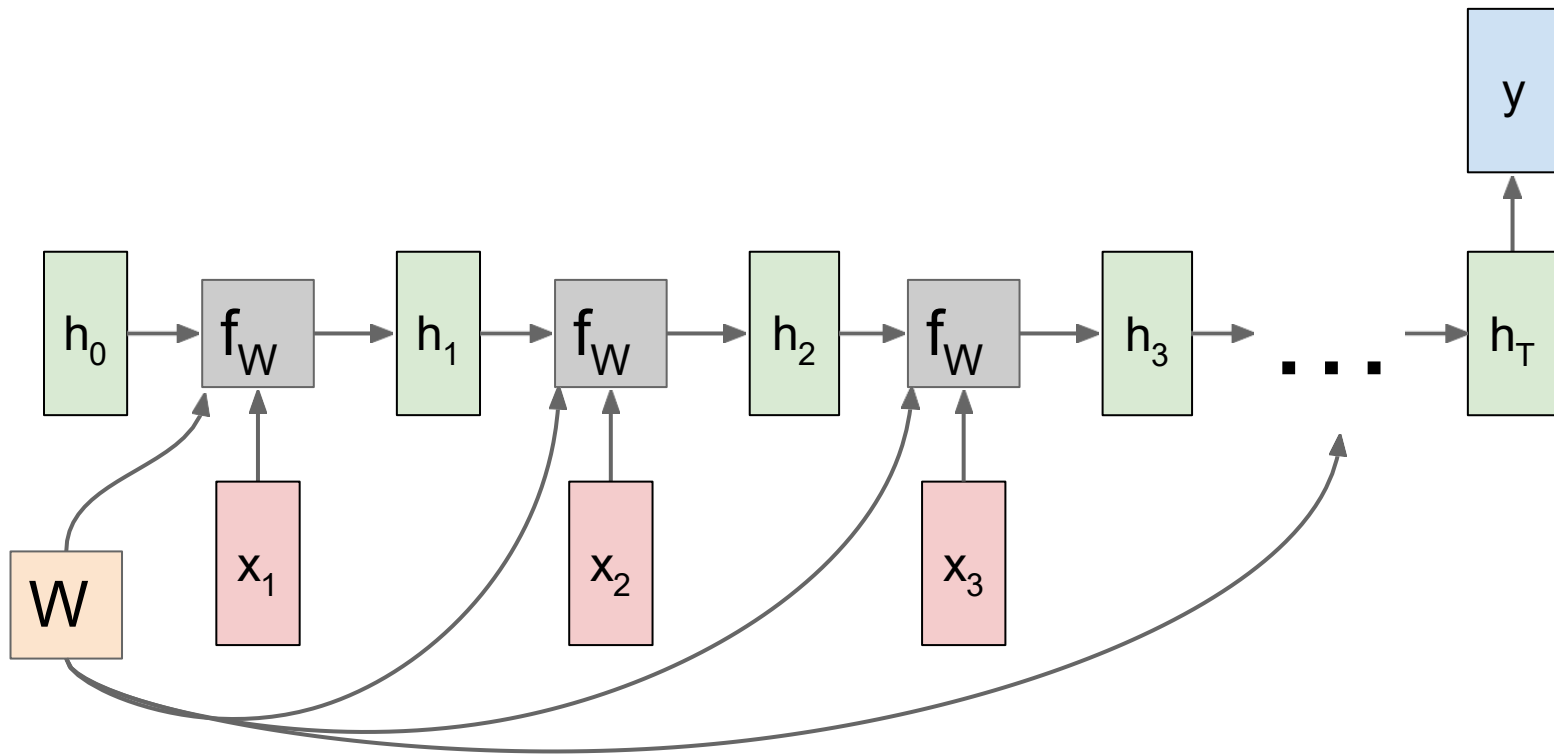
RNN: Computational Graph: Many to Many



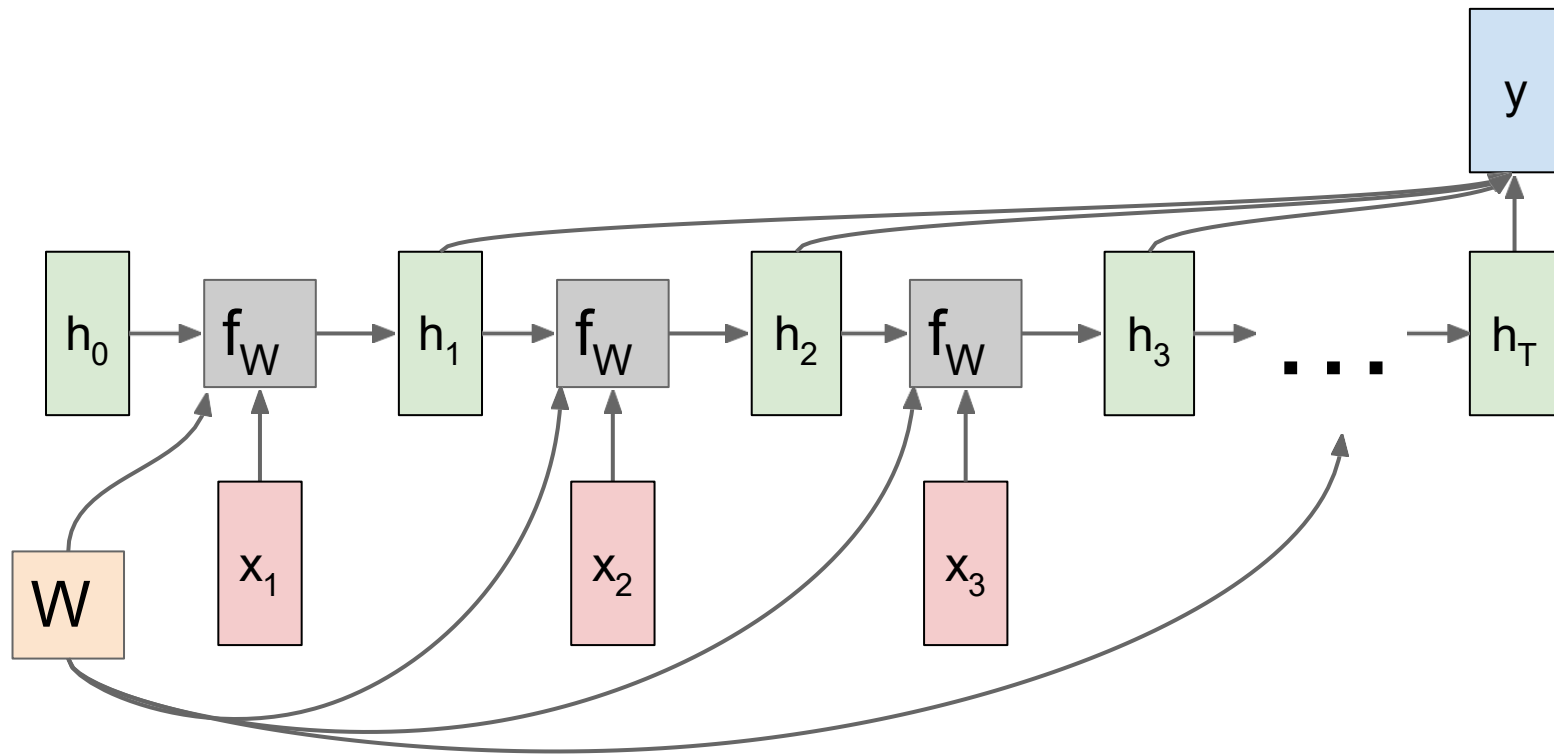
RNN: Computational Graph: Many to Many



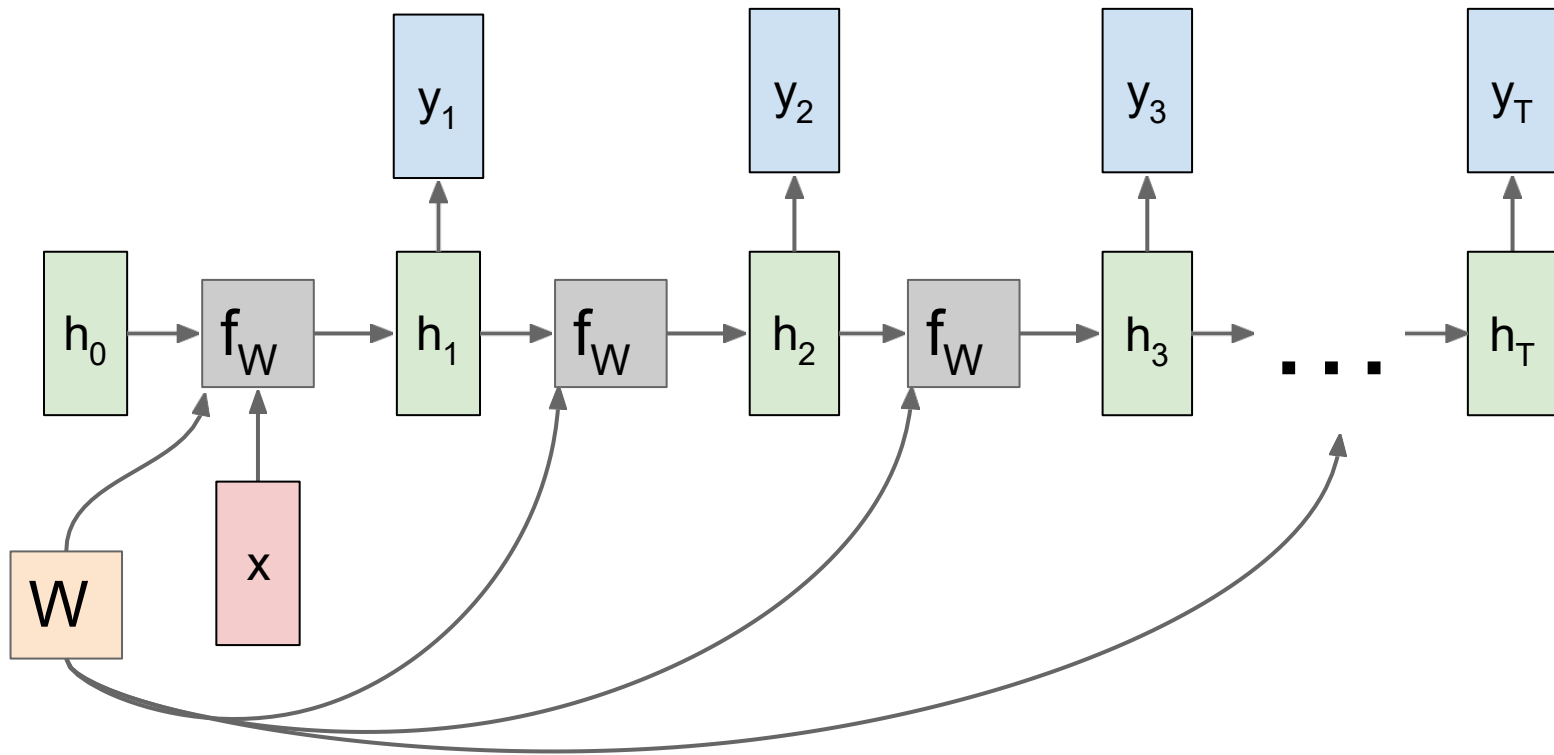
RNN: Computational Graph: Many to One



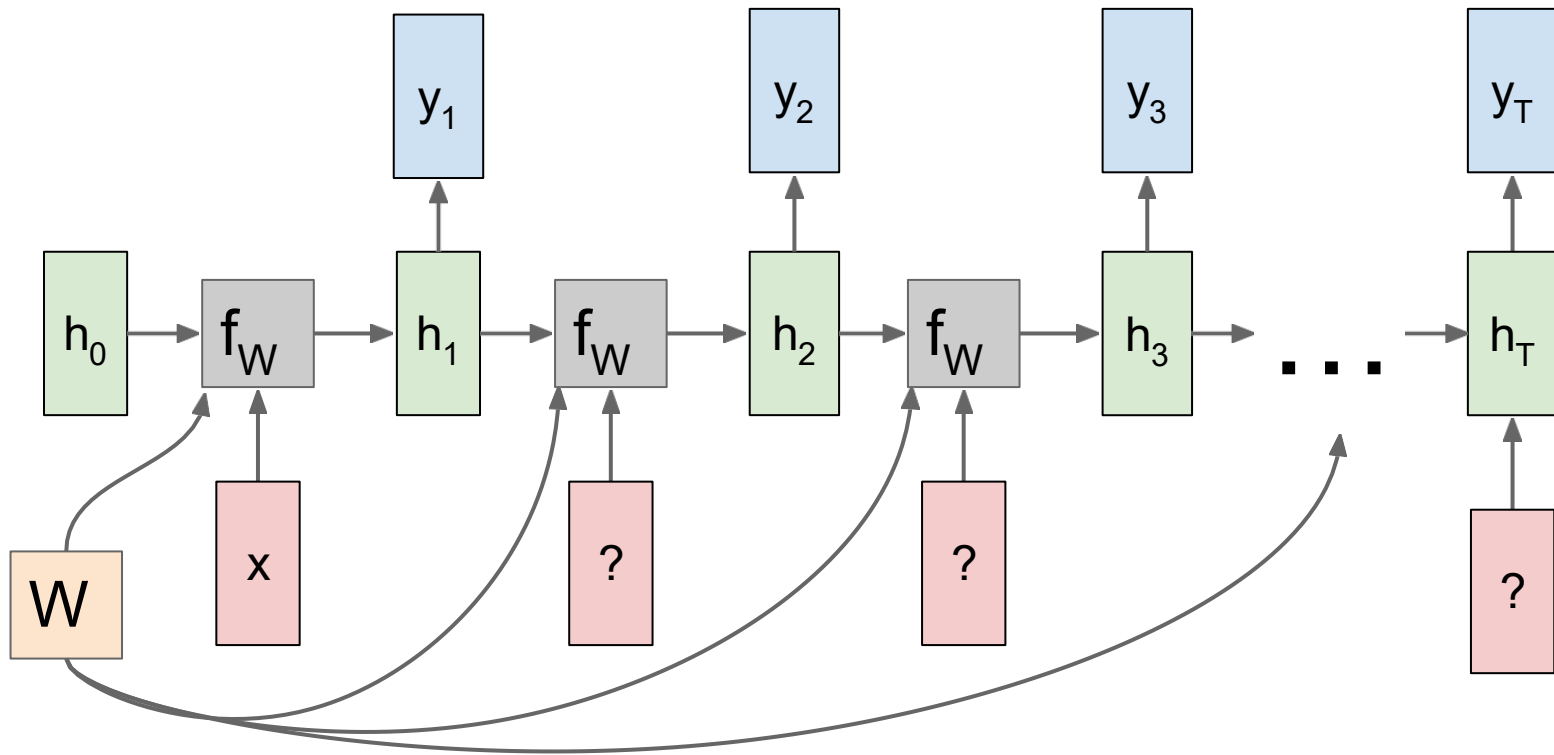
RNN: Computational Graph: Many to One



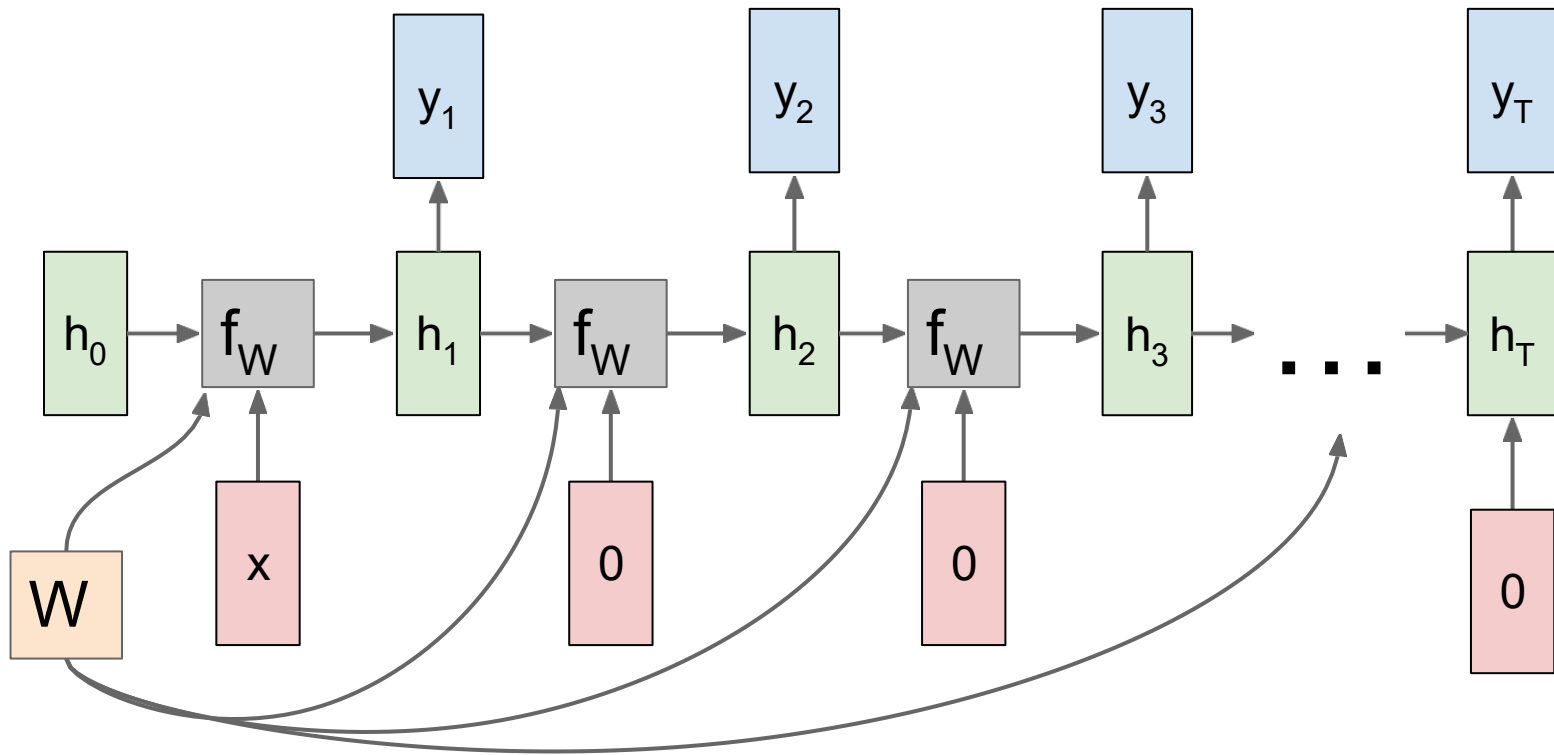
RNN: Computational Graph: One to Many



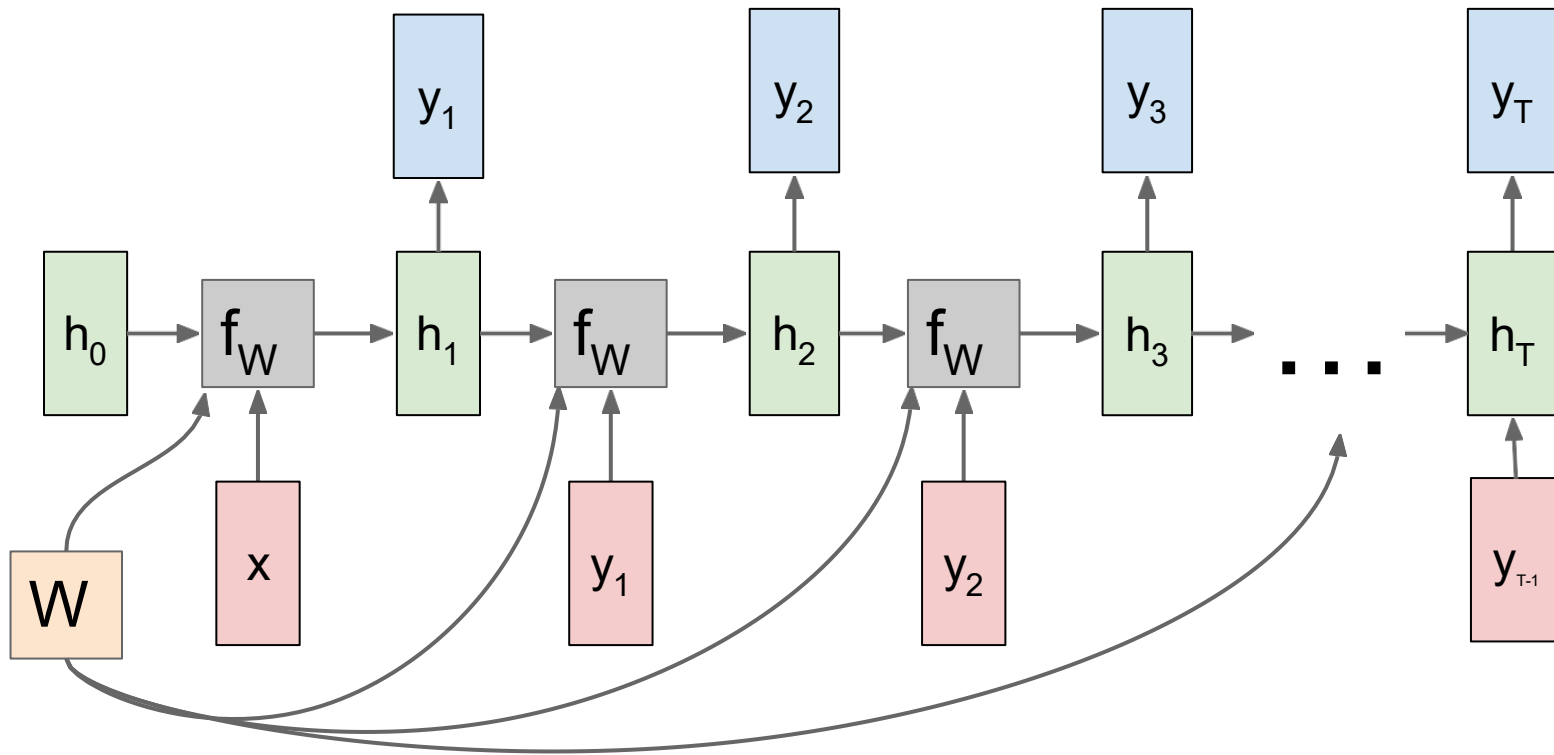
RNN: Computational Graph: One to Many



RNN: Computational Graph: One to Many

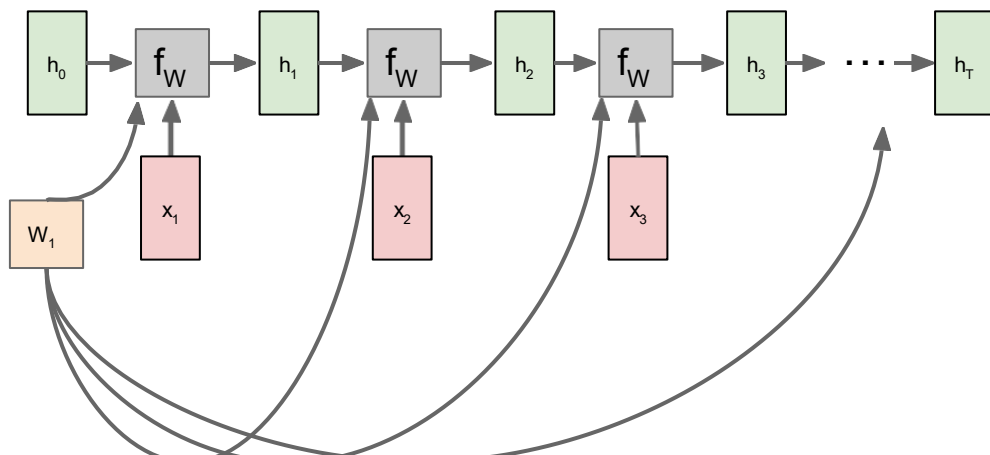


RNN: Computational Graph: One to Many



Sequence to Sequence: Many-to-one + one-to-many

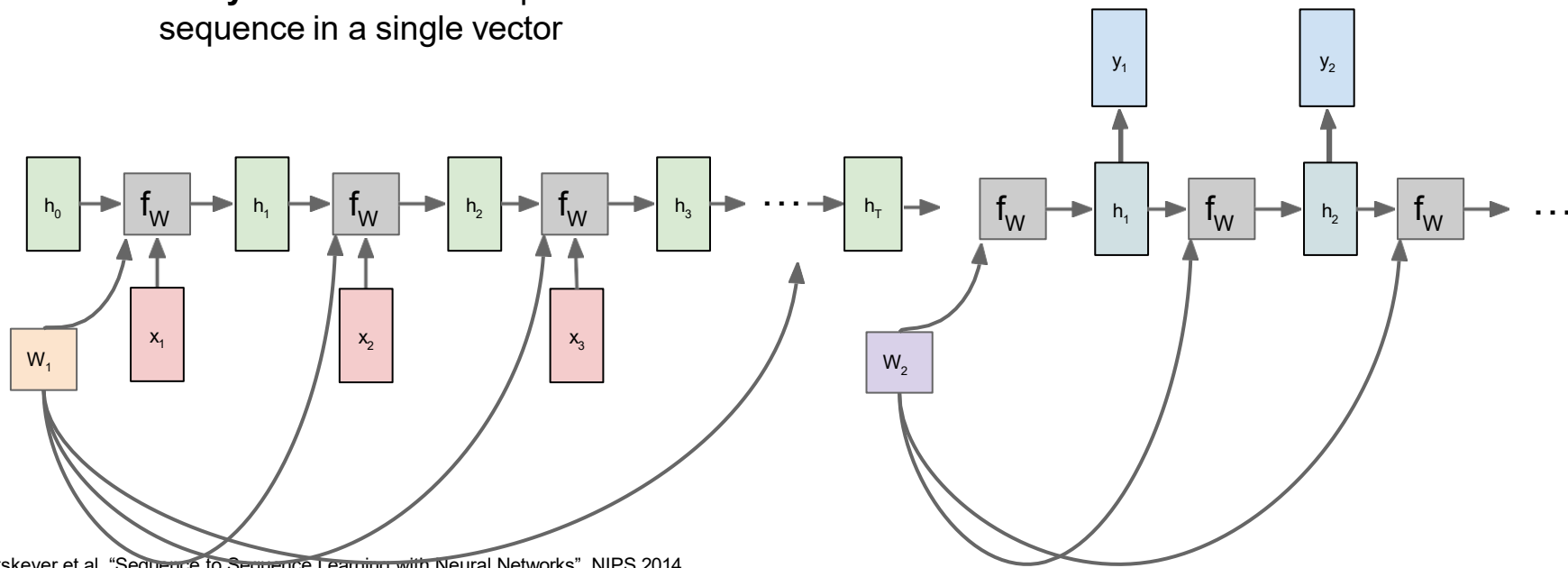
Many to one: Encode input sequence in a single vector



Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector

One to many: Produce output sequence from single input vector



Sequence Modelling: Design criteria

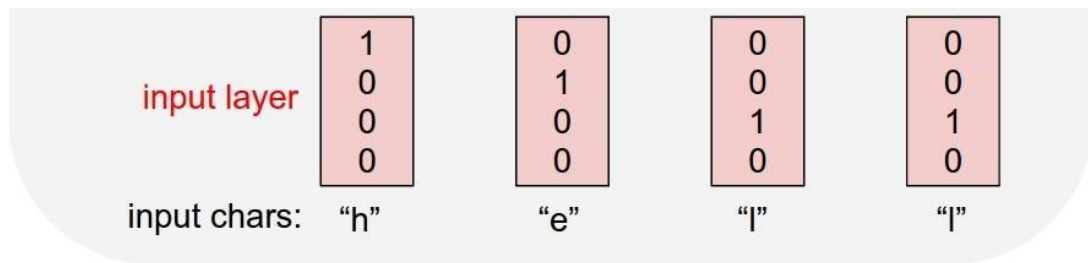
To model sequences, we need to:

1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence

Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

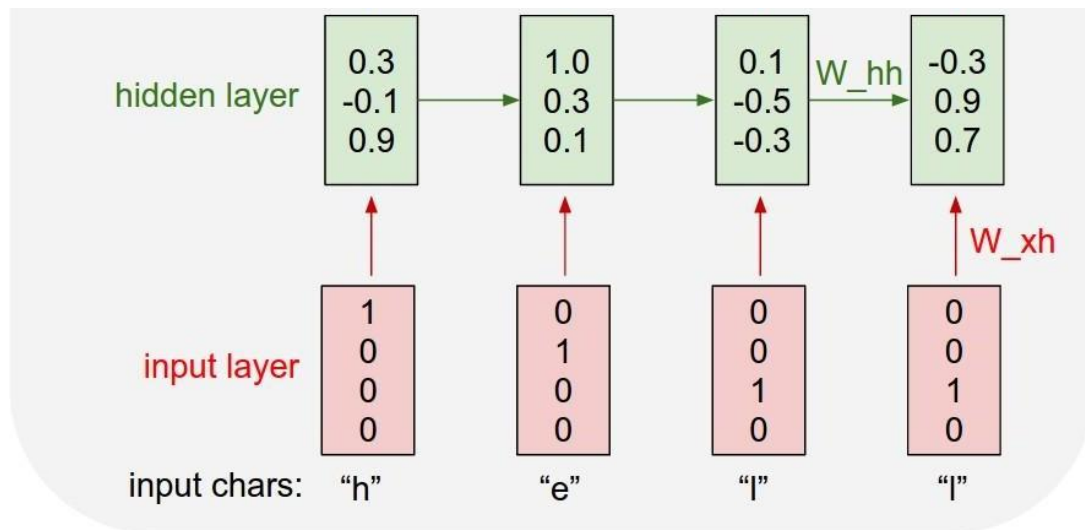


Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

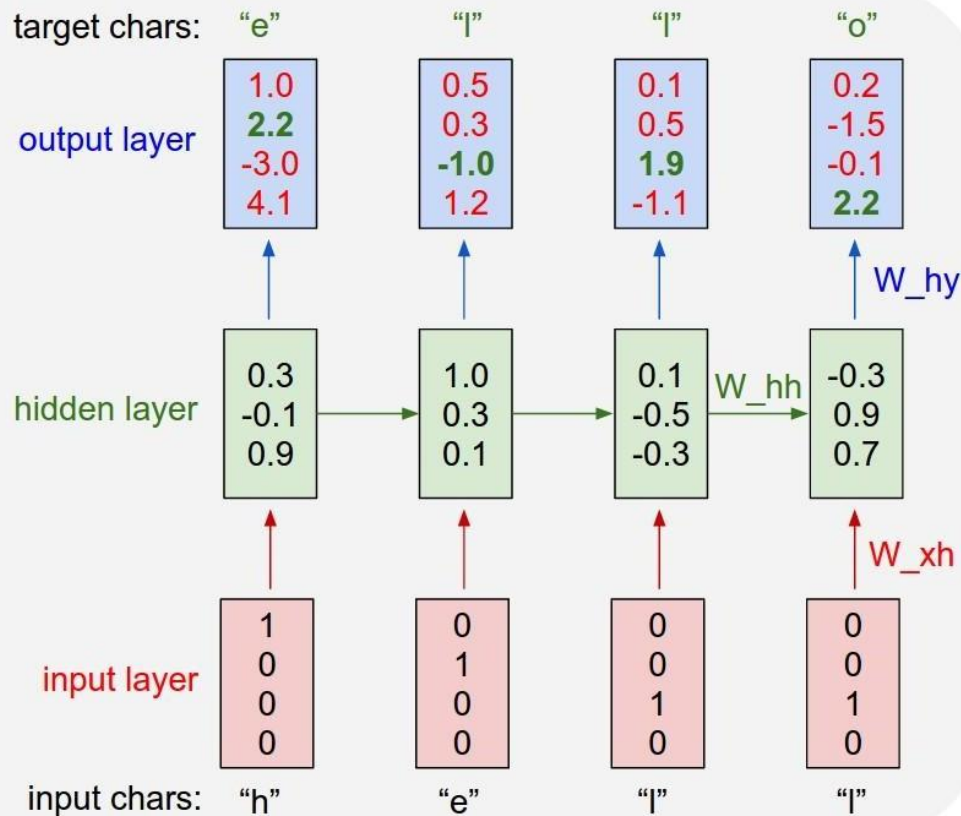
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

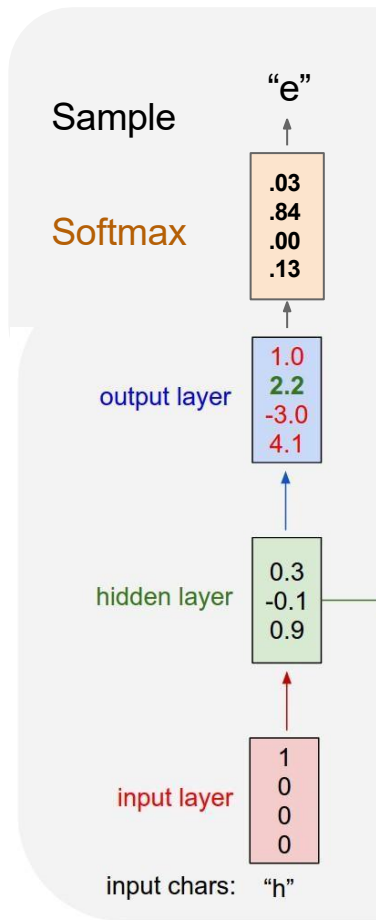
Example training
sequence:
“hello”



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

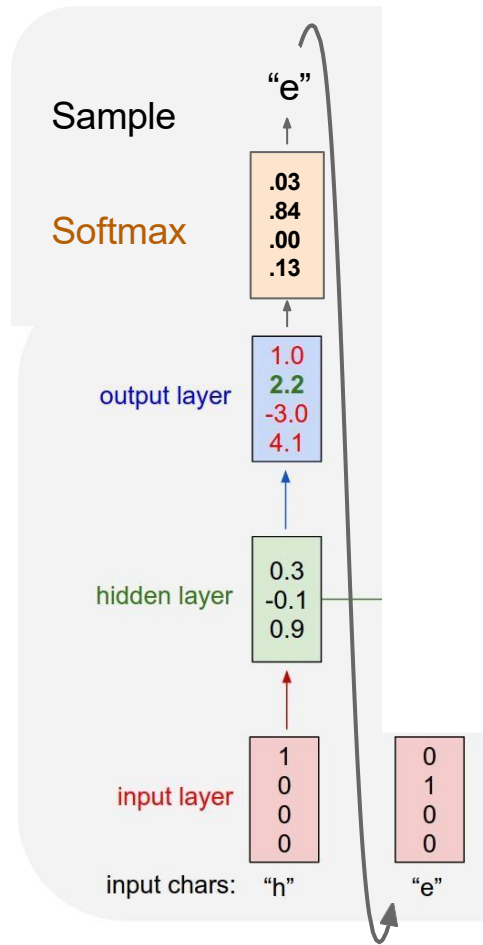
At test-time sample characters
one at a time, feed back to
model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

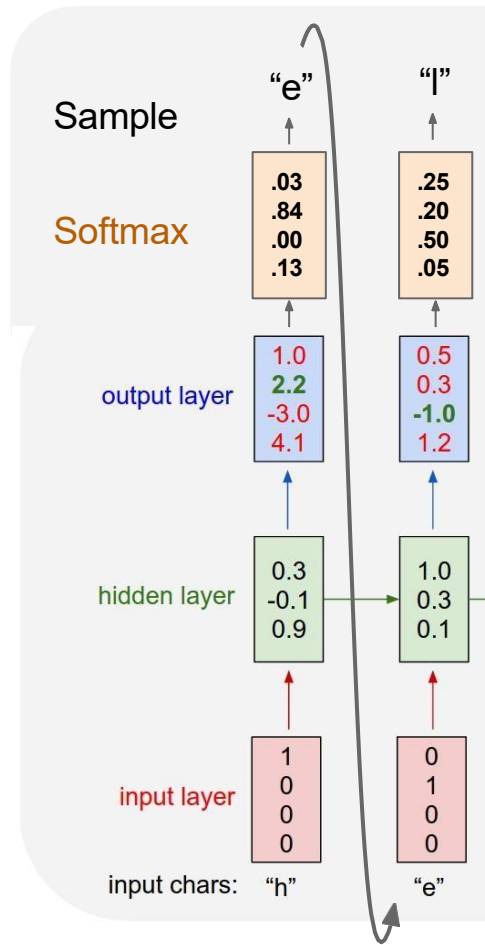
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

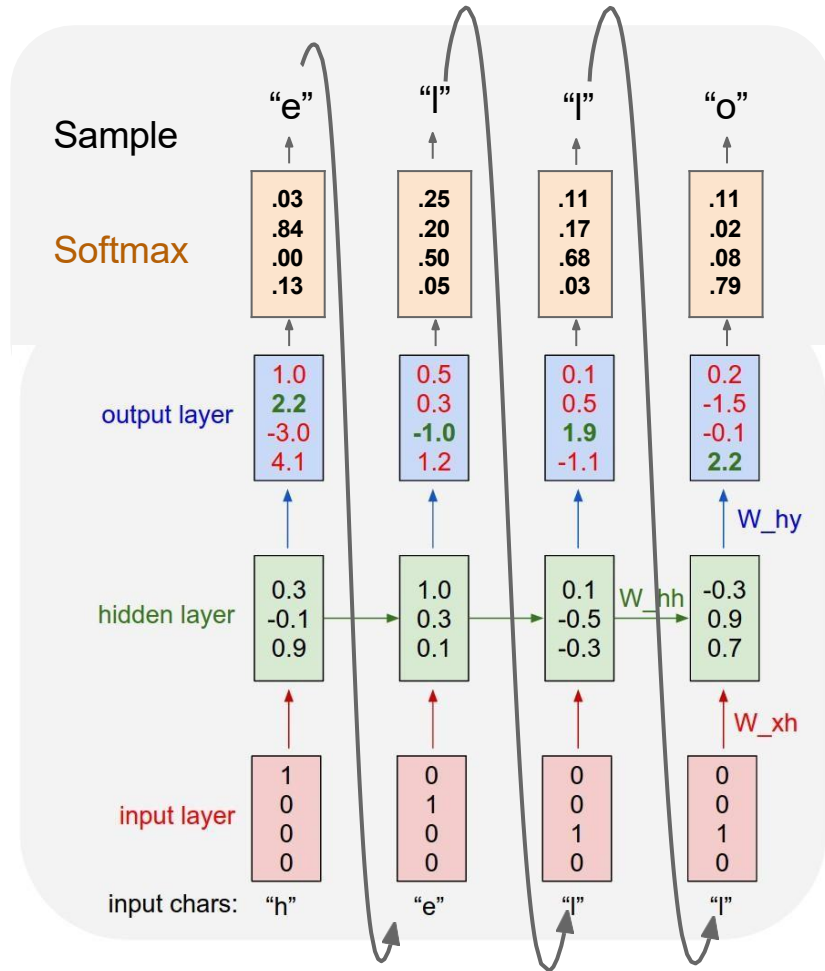
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model Sampling

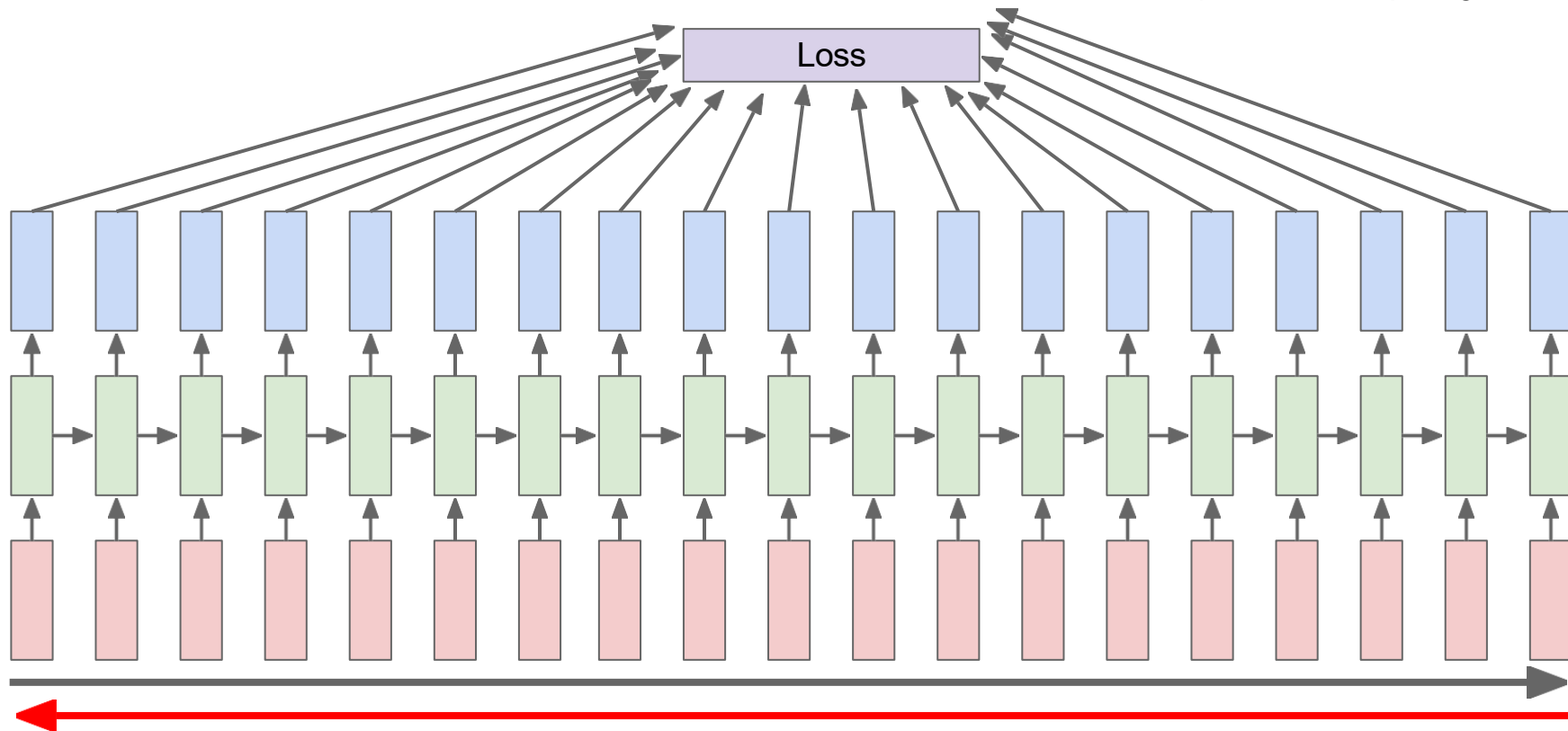
Vocabulary:
[h,e,l,o]

At test-time sample characters one at a time, feed back to model

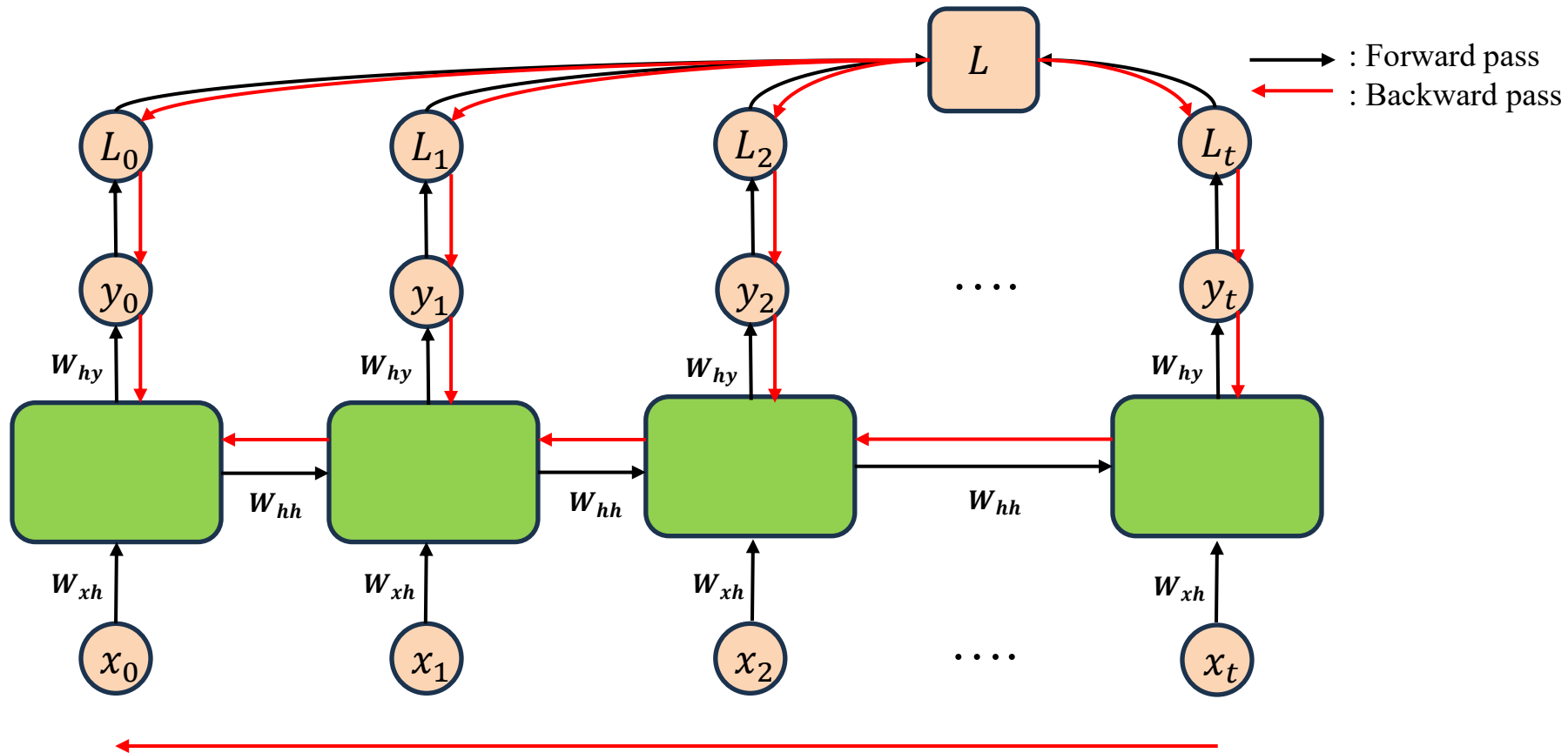


Backpropagation through time

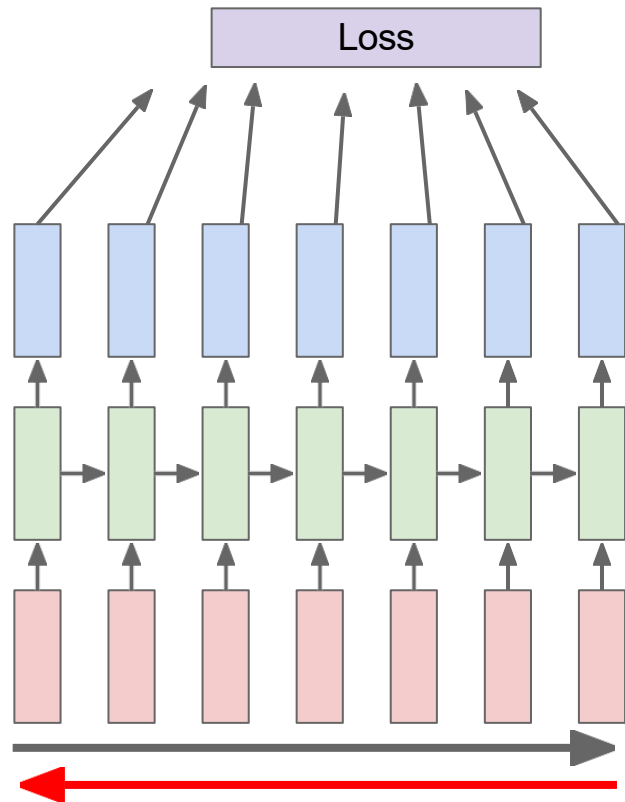
Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



RNNs: Backpropagation Through Time

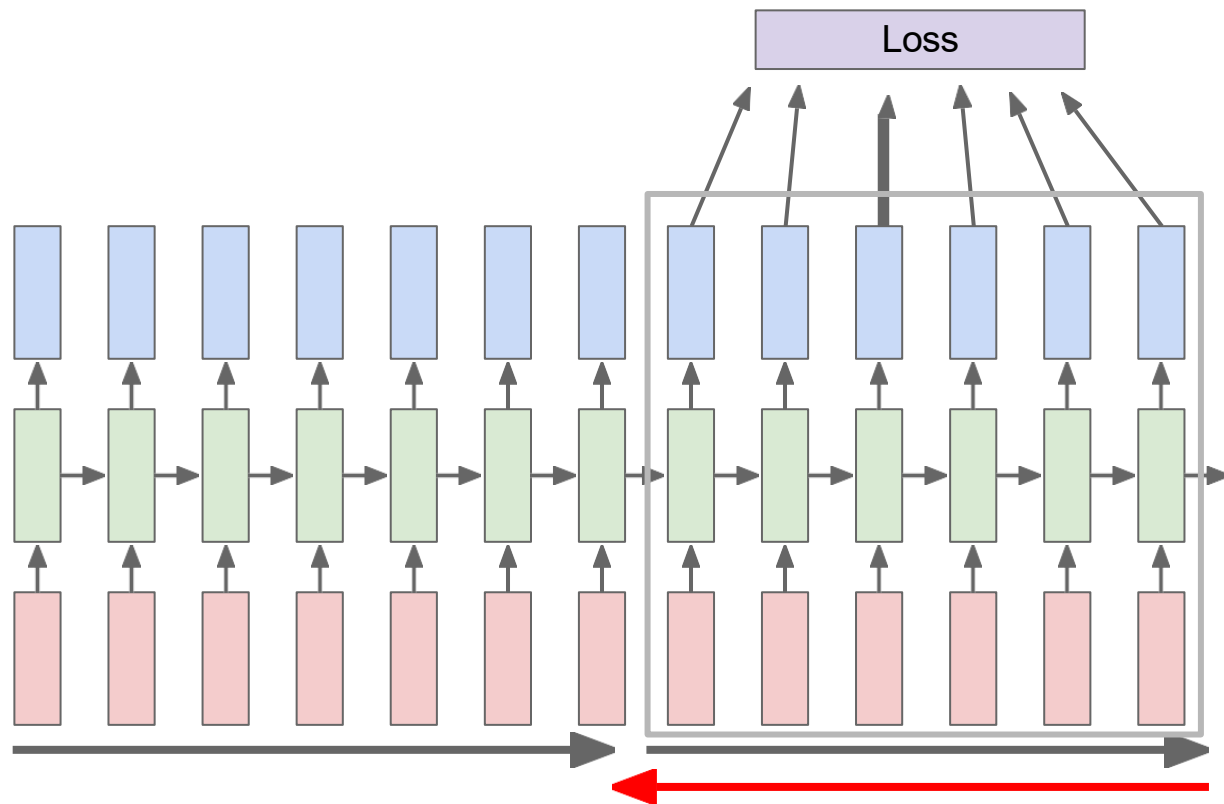


Truncated Backpropagation through time



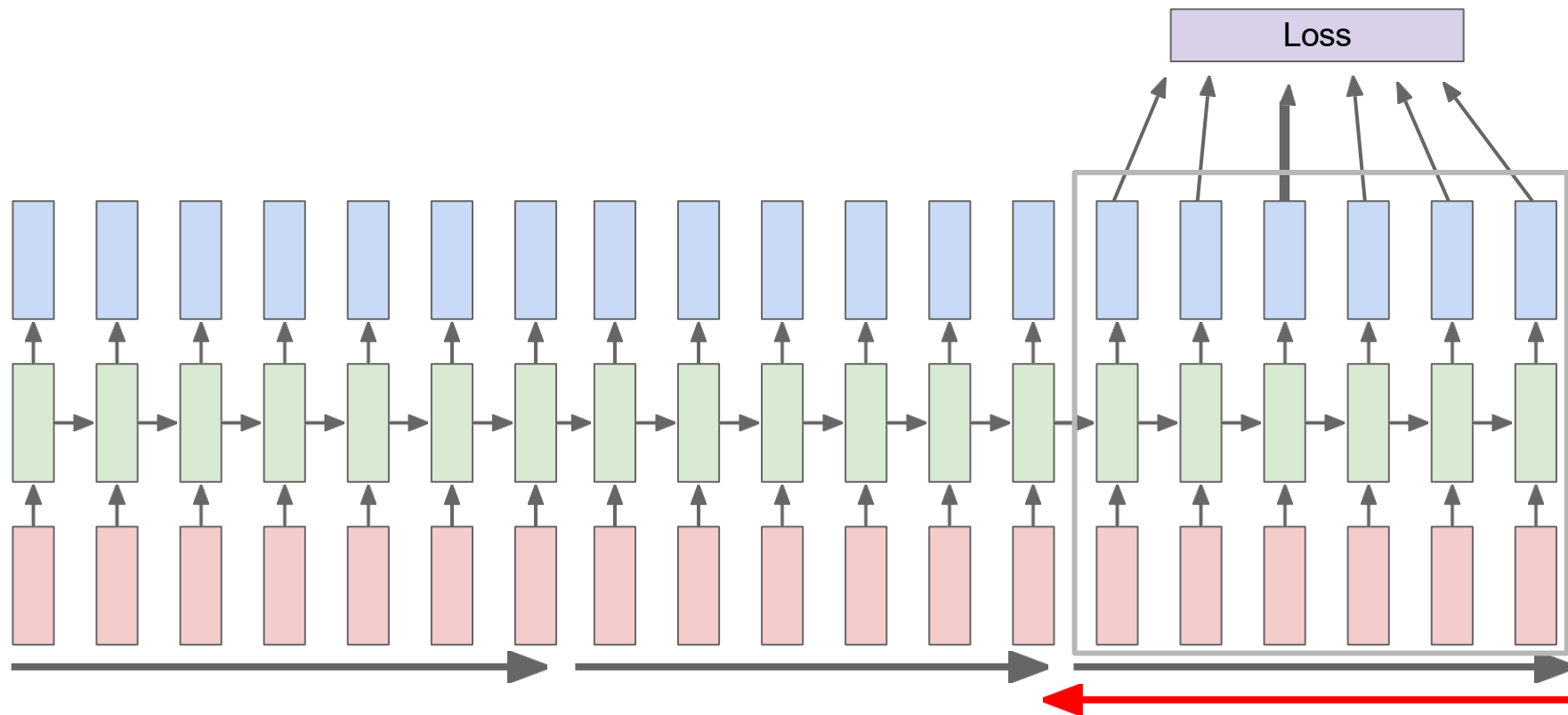
Run forward and backward through chunks of the sequence instead of whole sequence

Truncated Backpropagation through time



Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

Truncated Backpropagation through time



RNN tradeoffs

RNN Advantages:

- Can process any length input
- Computation for step t can (in theory) use information from many steps back
- Model size doesn't increase for longer input
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

Image Captioning: Example Results

Captions generated using [neuraltalk2](#)

All images are [CC0 Public domain](#)

[cat suitcase](#) [cat tree](#) [dog bear](#)

[surfers](#) [tennis](#) [giraffe](#) [motorcycle](#)



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

Image Captioning: Failure Cases

Captions generated using [neuraltalk2](#)
All images are [CC0 Public domain](#): [fur coat](#), [handstand](#), [spider web](#), [baseball](#)



A woman is holding a cat in her hand



A woman standing on a beach holding a surfboard



A bird is perched on a tree branch

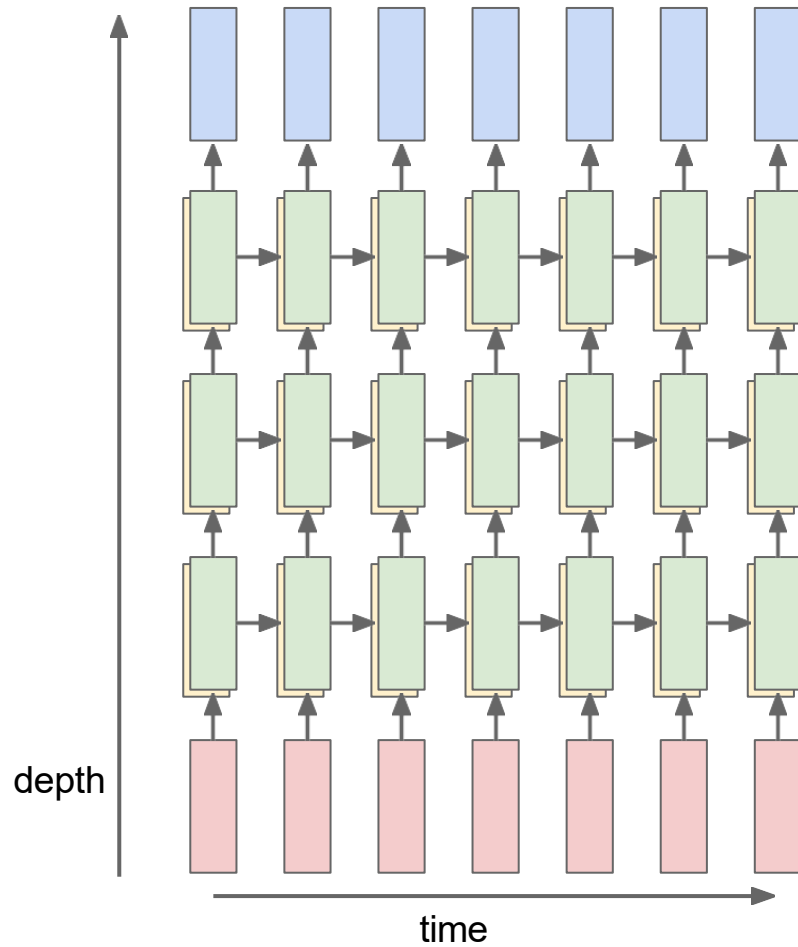


A person holding a computer mouse on a desk



A man in a baseball uniform throwing a ball

Multilayer RNNs



Challenges in Training an RNN:

Many values > 1
Exploding Gradients
Solution:
Gradient clipping

Many values < 1
Vanishing Gradients
Solution:

1. **Activation function**
2. **Weight initialization**
3. **Network architecture**

The Problem of Long-Term Dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together



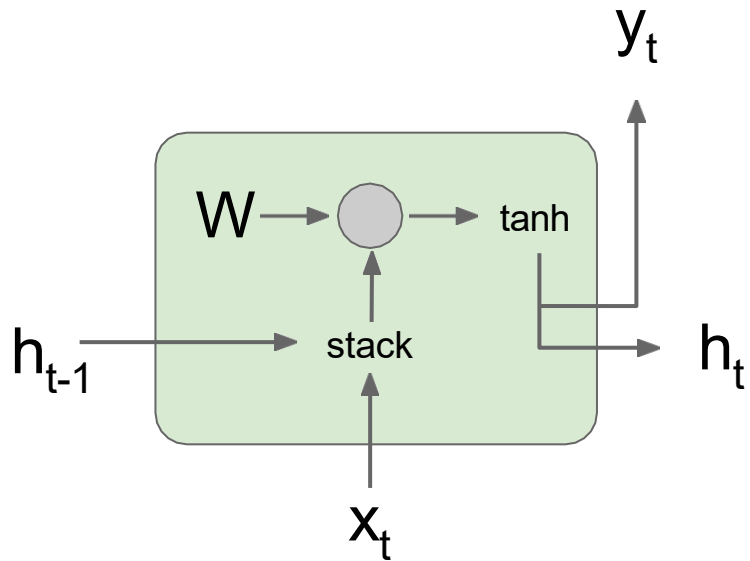
Errors due to further back time steps have smaller and smaller gradients



Bias parameters to capture short-term dependencies

Vanilla RNN Gradient Flow

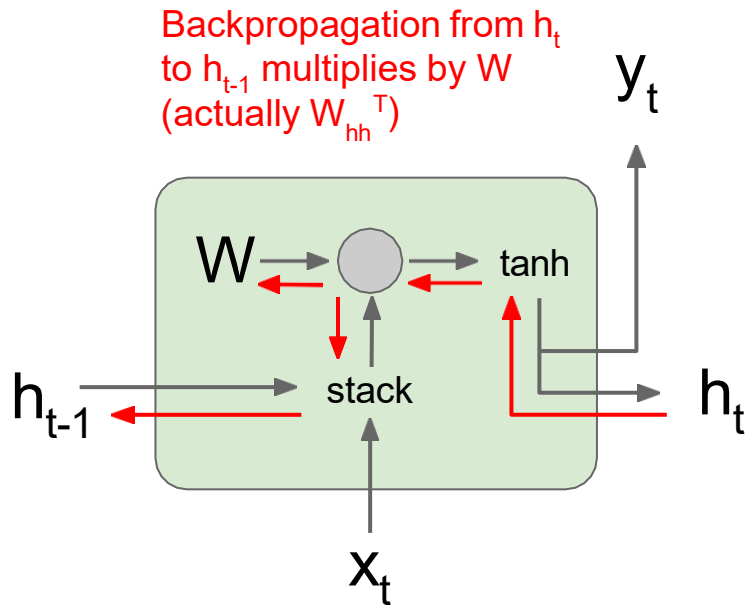
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

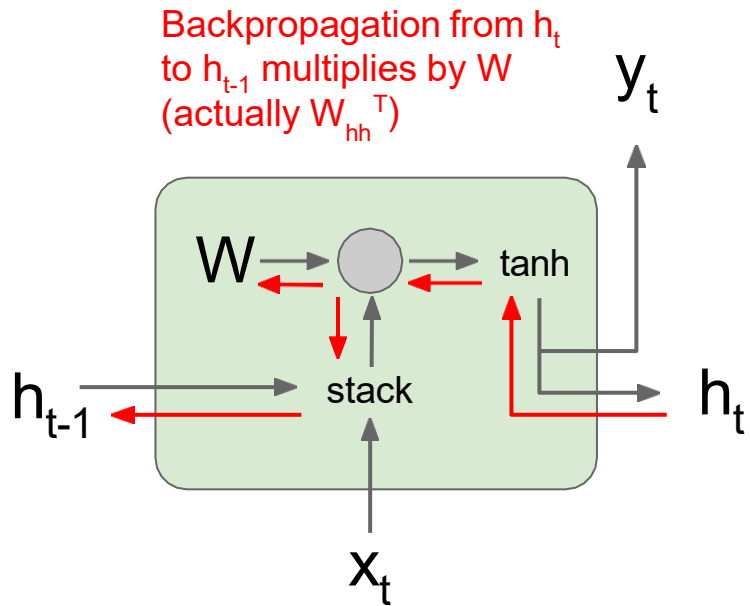
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\&= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\&= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)\end{aligned}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

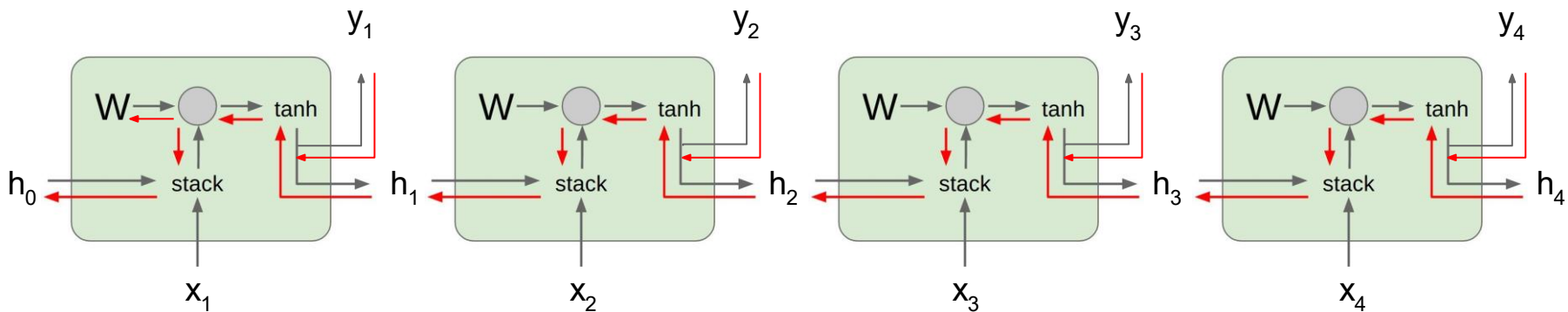


$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\&= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\&= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)\end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

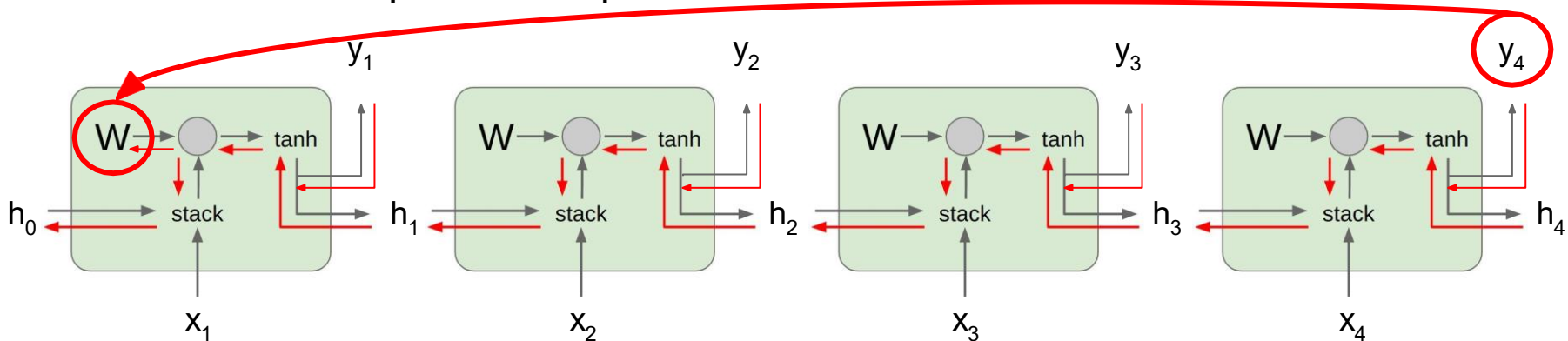


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



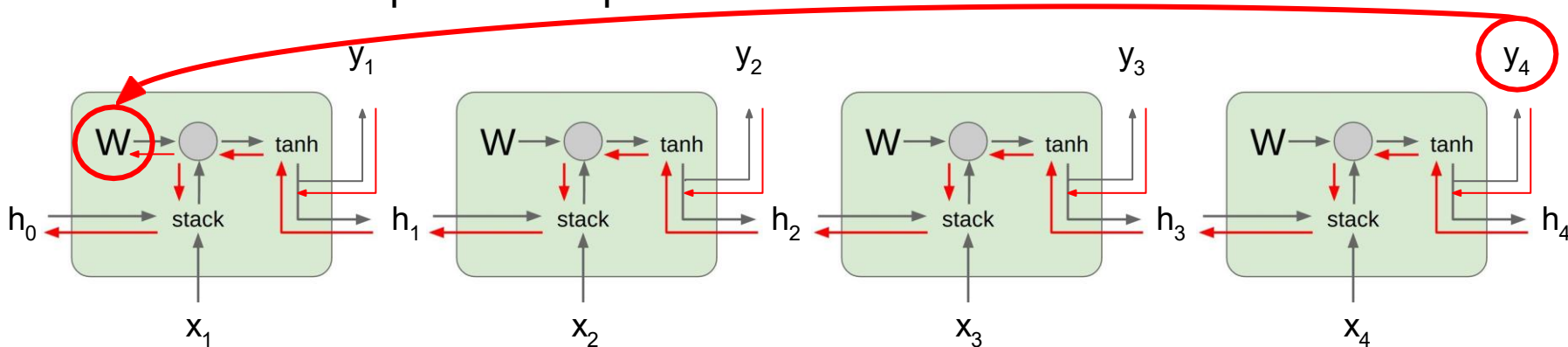
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



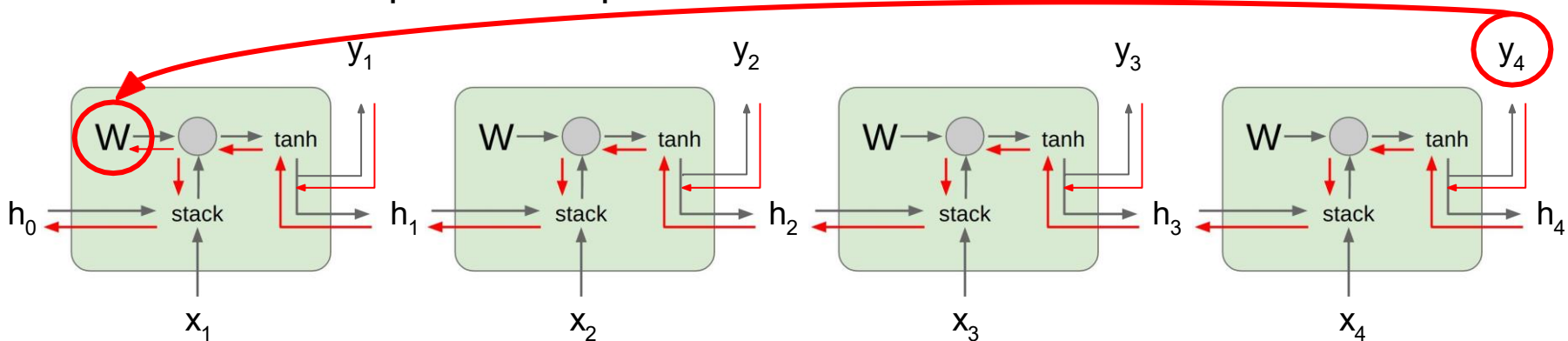
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

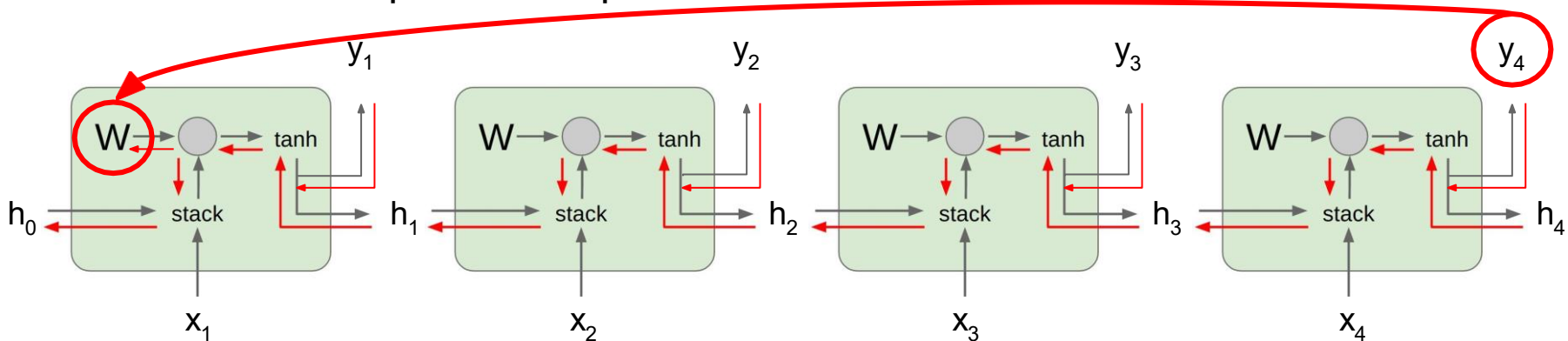
$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

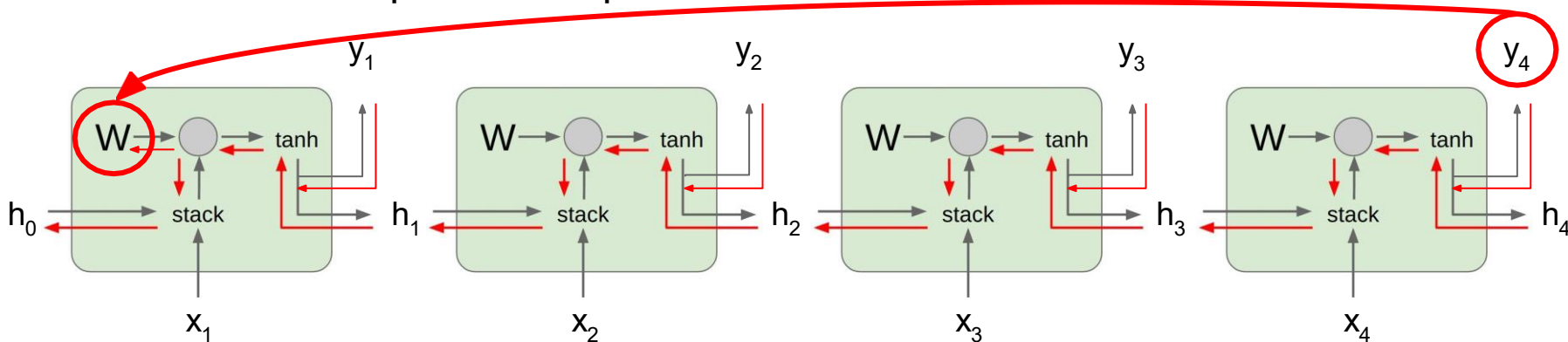
Almost always < 1
Vanishing gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \boxed{\tanh'(W_{hh} h_{t-1} + W_{xh} x_t)} \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



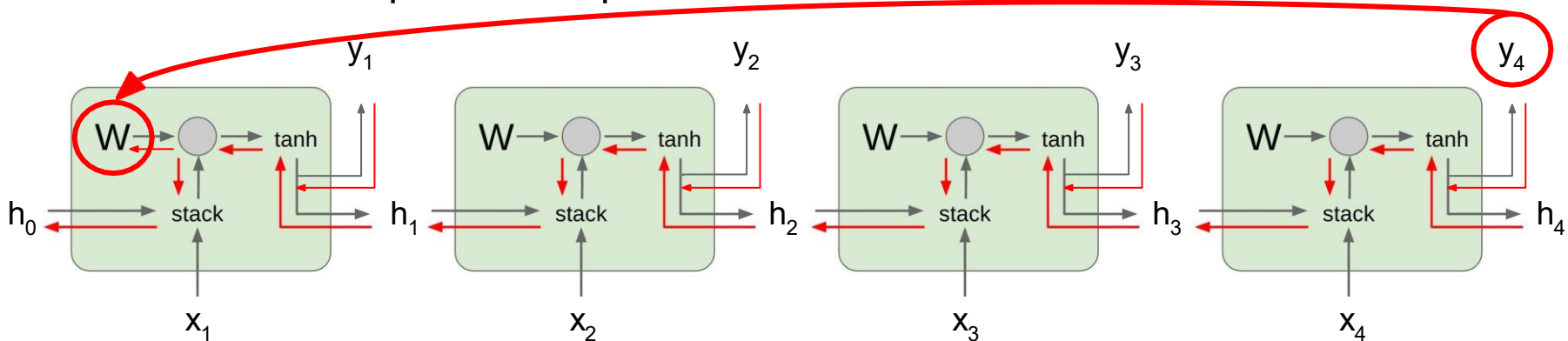
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

What if we assumed no non-linearity?

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1 :
Exploding gradients

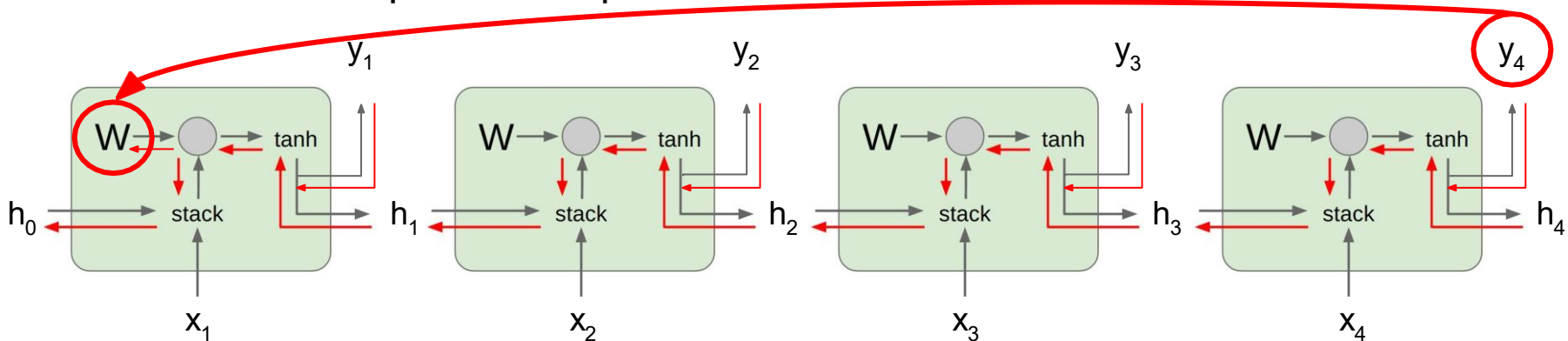
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1 :
Vanishing gradients

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value > 1 :
Exploding gradients

Largest singular value < 1 :
Vanishing gradients

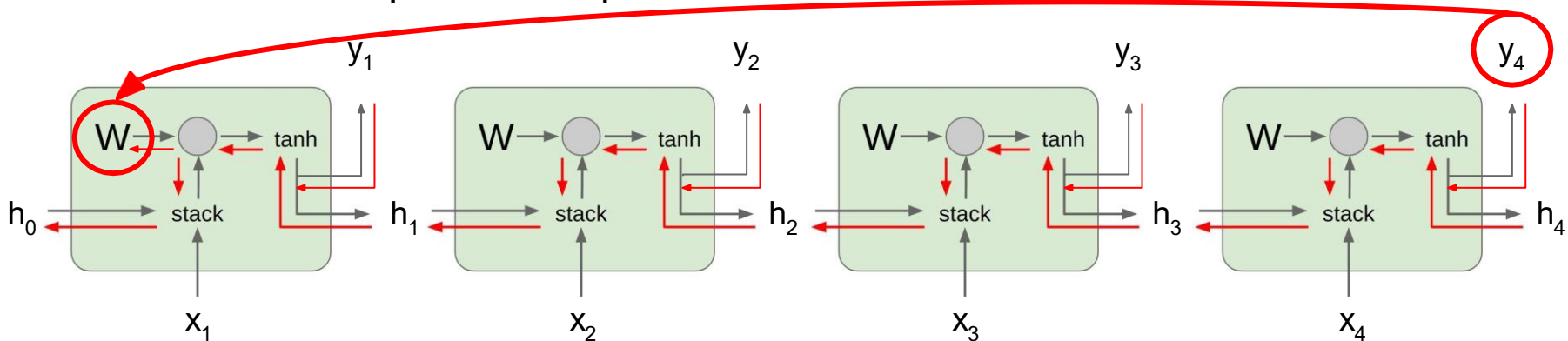
→ **Gradient clipping:**
Scale gradient if its norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

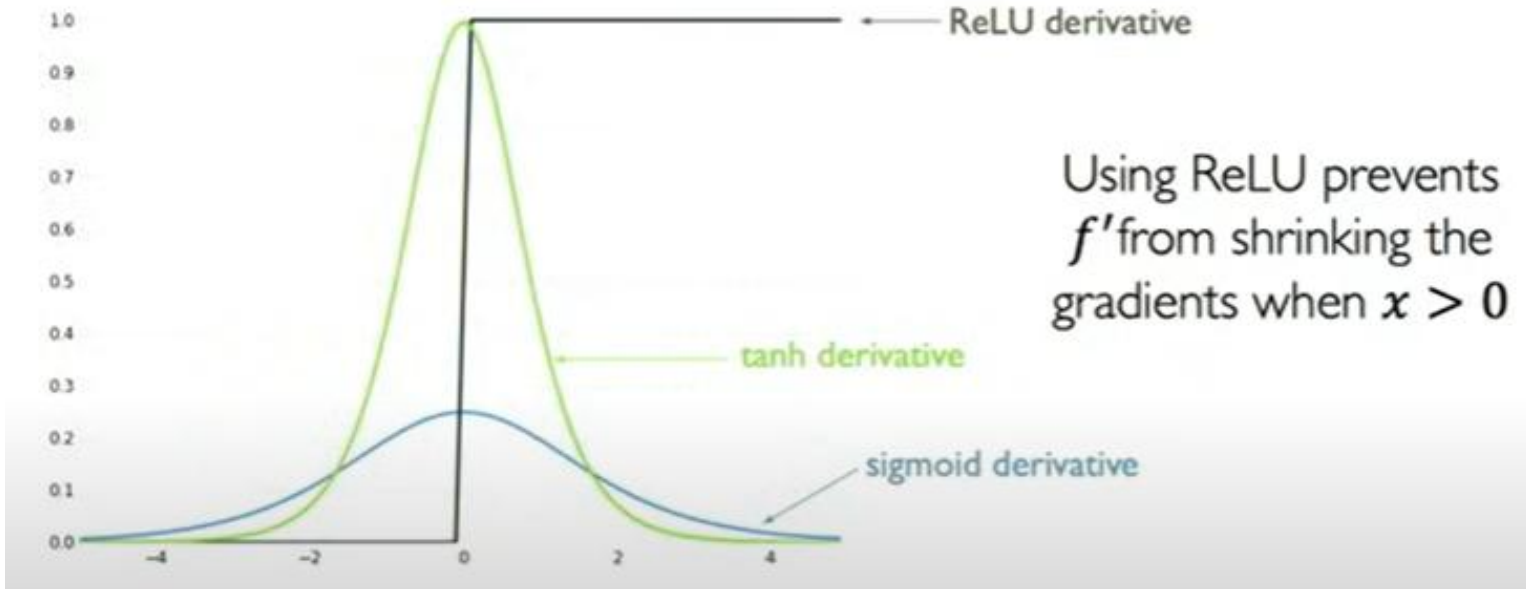
Largest singular value > 1:
Exploding gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1:
Vanishing gradients

→ Change RNN architecture

Solution #1: Activation Function



Solution #2: Parameter Initialization

Initialize **weights** to identity matrix

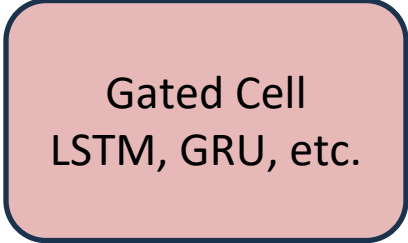
Initialize **biases** to zero

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}$$

This helps prevent the weights from shrinking to zero.

Solution #3: Gated Cells

Idea: Use **gates** to selectively **add** or **remove** information within **each recurrent unit with**



Gated Cell
LSTM, GRU, etc.

Gates optionally let information through the cell

Long Short Term Memory(LSTMs) networks rely on a gated cell to track information throughout many time steps.

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state

$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state

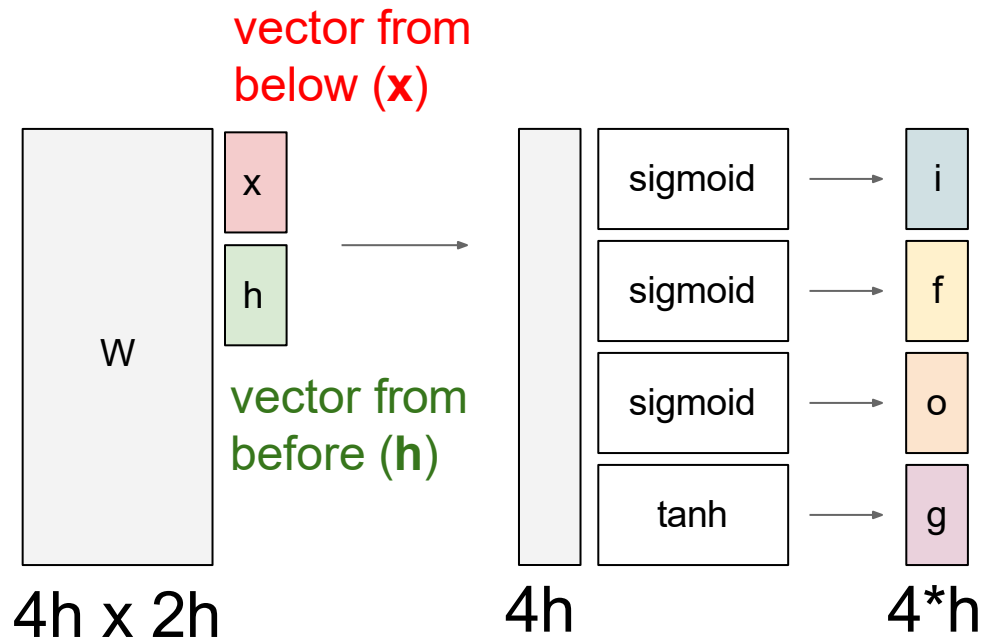
$$h_t = o \odot \tanh(c_t)$$

LSTMs: Key concepts

1. Maintain a **cell state**
2. Use **gates** to control the **flow of information**
 - **Forget** gate gets rid of irrelevant information
 - **Store** relevant information from current input
 - Selectively **update** cell state
 - **Output** gate returns a filtered version of the cell state
3. Backpropagation through time with partially **uninterrupted gradient flow**

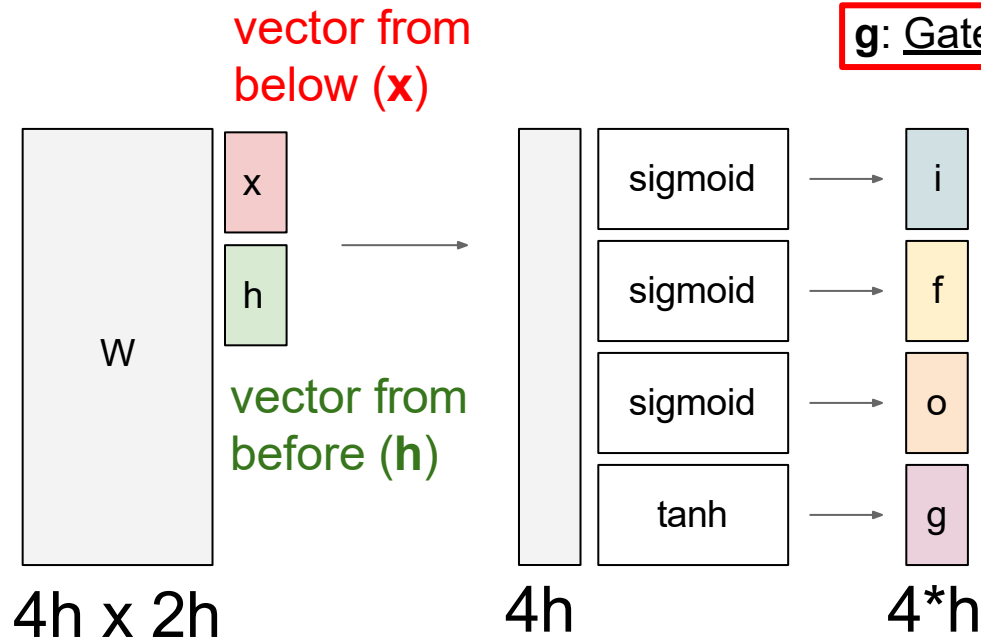
Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



g: Gate gate (?), How much to write to cell

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

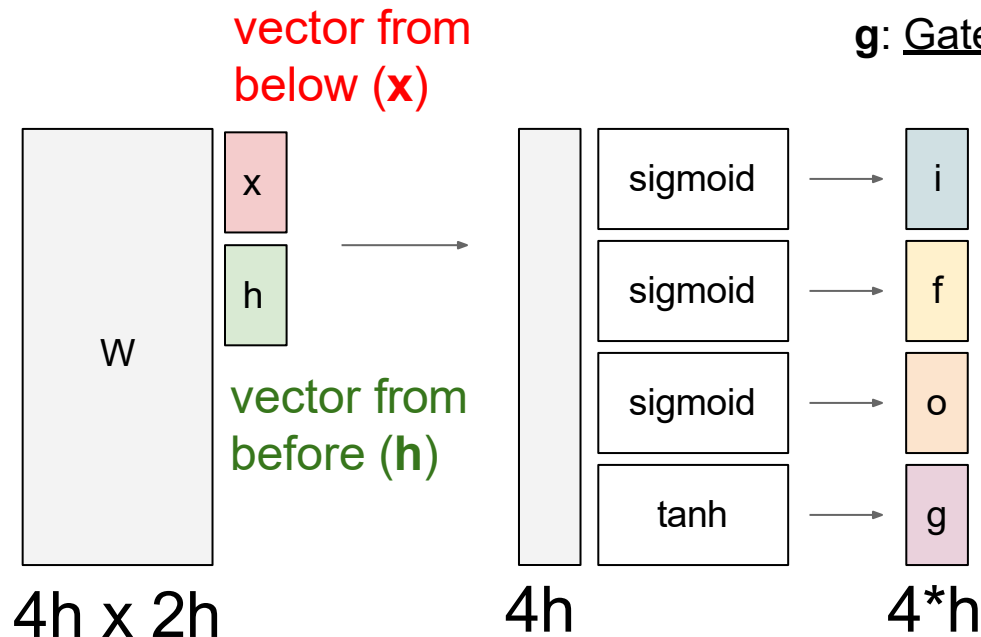
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

i: Input gate, whether to write to cell



g: Gate gate (?), How much to write to cell

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

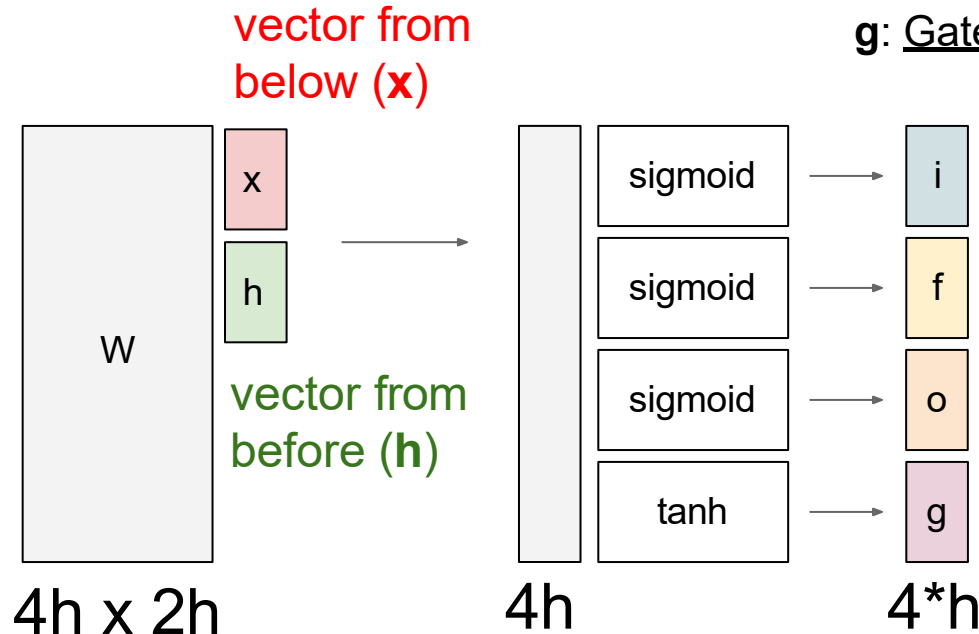
Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

i: Input gate, whether to write to cell

f: Forget gate, Whether to erase cell

g: Gate gate (?), How much to write to cell



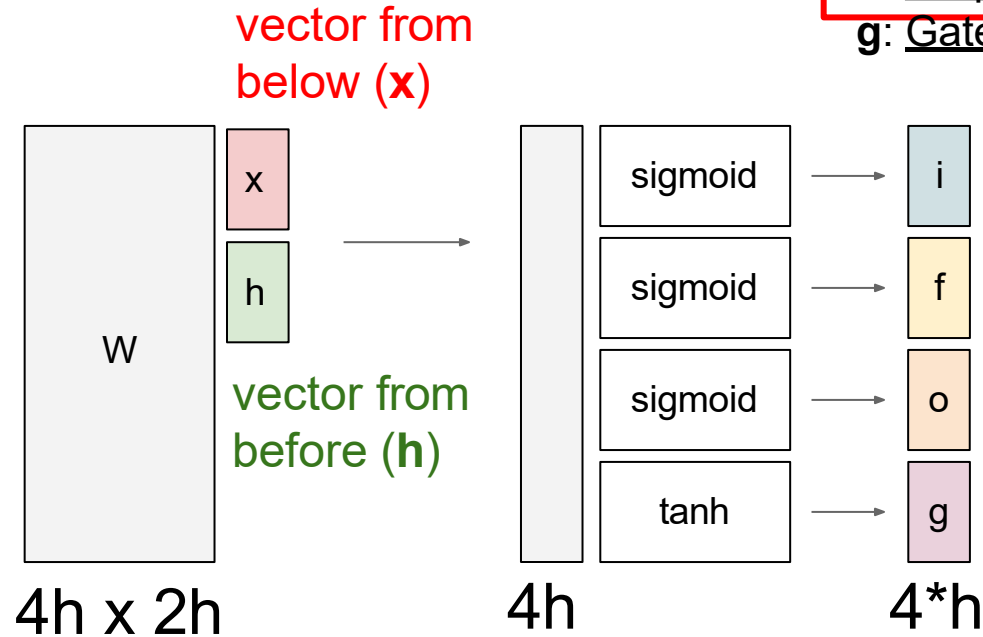
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



i : Input gate, whether to write to cell

f : Forget gate, Whether to erase cell

o : Output gate, How much to reveal cell

g : Gate gate (?), How much to write to cell

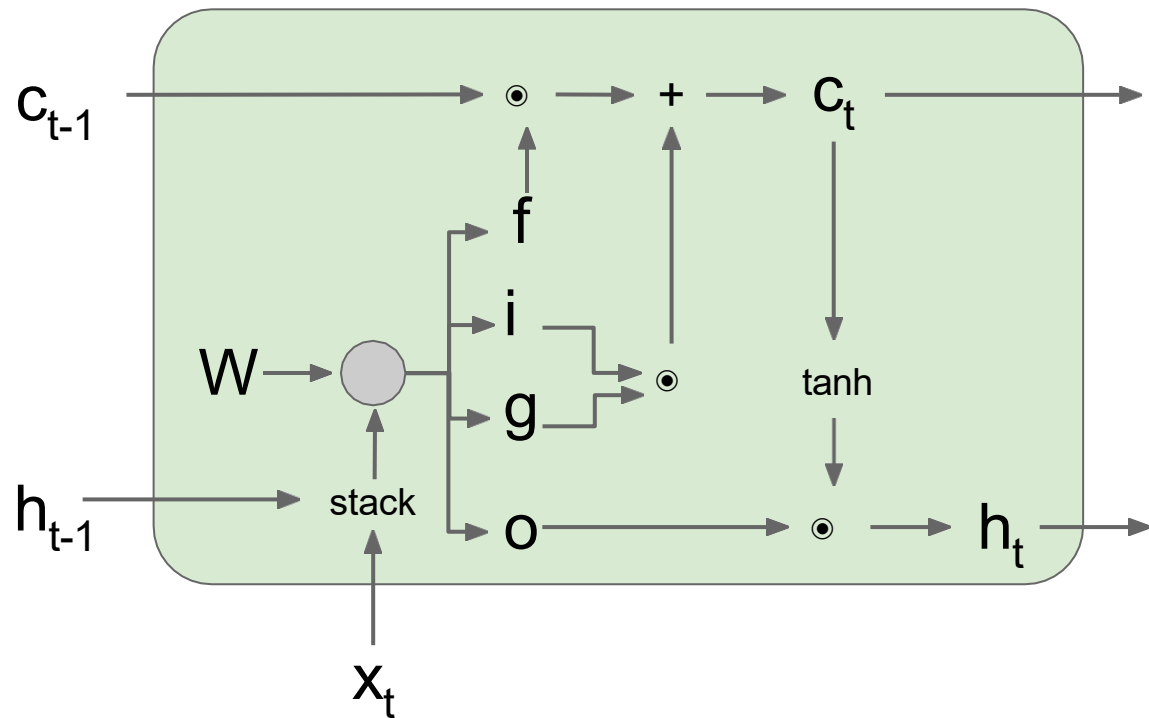
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

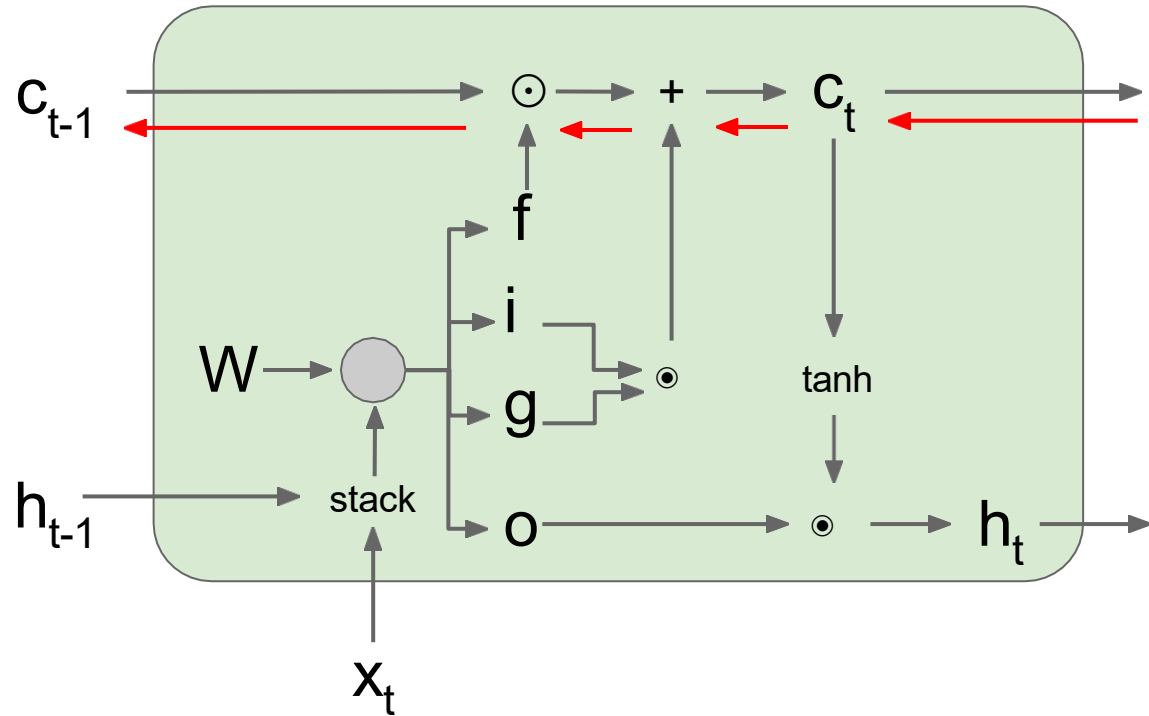
[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]



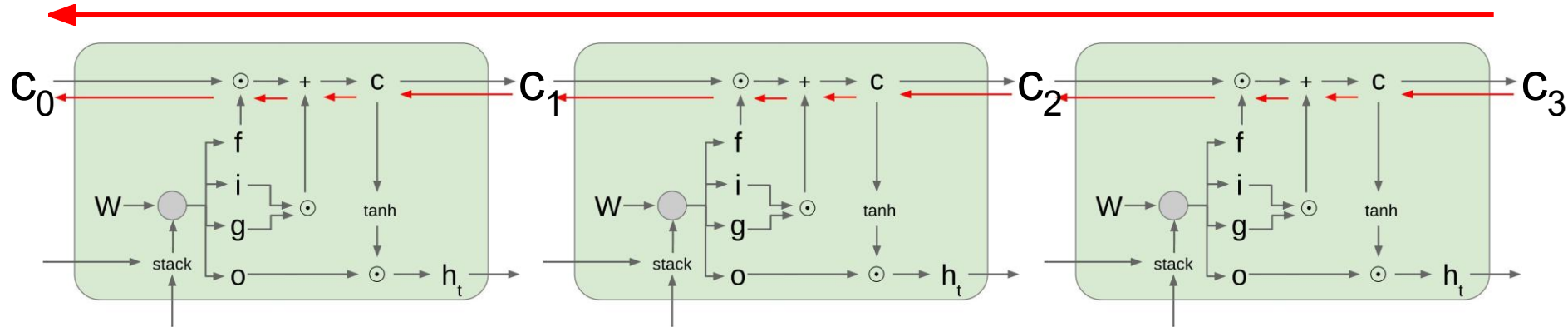
Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]

Uninterrupted gradient flow!



Notice that the gradient contains the **f** gate's vector of activations

- allows better control of gradients values, using suitable parameter updates of the forget gate.

Also notice that are added through the **f**, **i**, **g**, and **o** gates

- better balancing of gradient values

Do LSTMs solve the vanishing gradient problem?

The LSTM architecture makes it easier for the RNN to preserve information over many timesteps

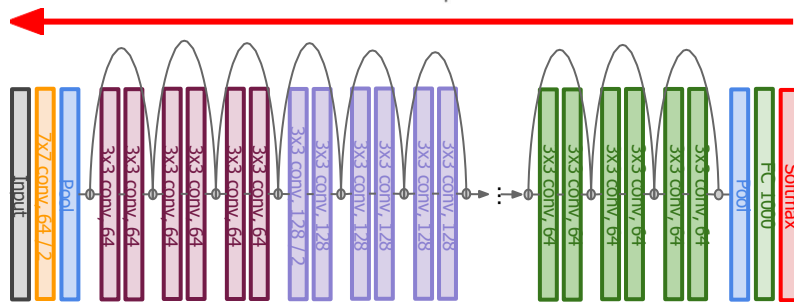
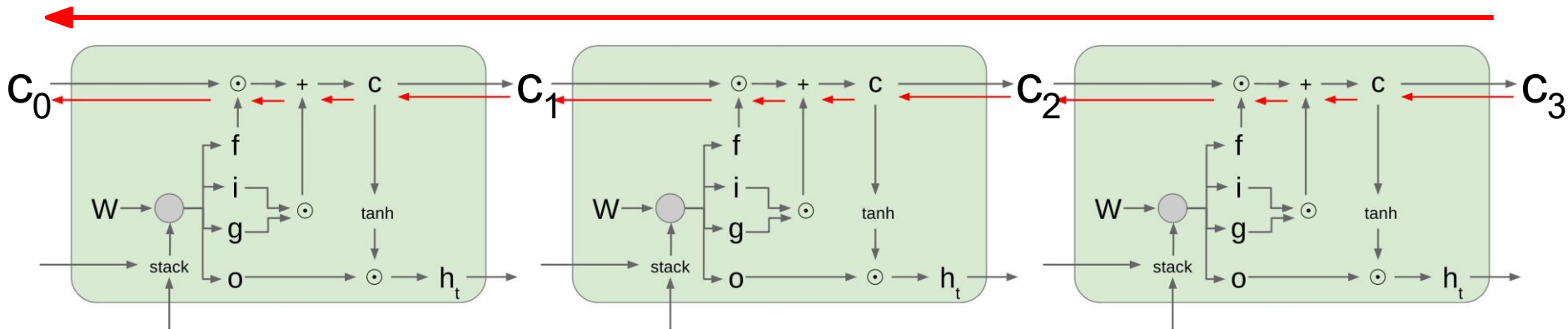
- e.g. **if the $f = 1$ and the $i = 0$** , then the information of that cell is preserved indefinitely.
- By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state

LSTM **doesn't guarantee** that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]

Uninterrupted gradient flow!



Similar to ResNet!

Long Short Term Memory (LSTM): Gradient Flow

Uninterrupted gradient flow!

Similar to ResNet!

In between:
Highway Networks

Srivastava et al, "Highway Networks",
ICML DL Workshop 2015

GRU and LSTM

Problems with RNN

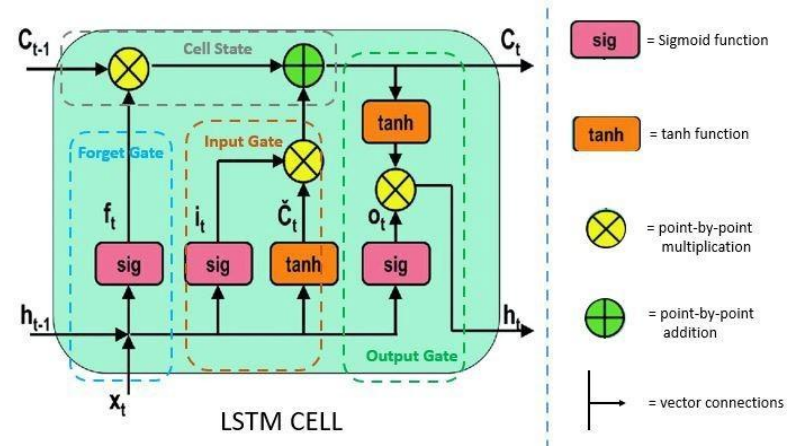
- Exploding and vanishing gradient problems during backpropagation.
- Gradients are those values which to update neural networks weights. In other words, we can say that Gradient carries information.
- Vanishing gradient is a big problem in deep neural networks. it vanishes or explodes quickly in earlier layers and this makes RNN unable to hold information of longer sequence and thus RNN becomes short-term memory.
- If we apply RNN for a paragraph RNN may leave out necessary information due to gradient problems and not be able to carry information from the initial time step to later time steps.

How to solve these problems?

- The reason for exploding gradient was the capturing of relevant and irrelevant information. a model which can decide what information from a paragraph is relevant and remember only relevant information and throw all the irrelevant information
- This is achieved by using gates. the LSTM (Long -short-term memory) and GRU (Gated Recurrent Unit) have gates as an internal mechanism, which control what information to keep and what information to throw out. By doing this LSTM, GRU networks solve the exploding and vanishing gradient problem.
- Almost each and every SOTA (state of the art) model based on RNN follows LSTM or GRU networks for prediction.

LSTM Networks

Every LSTM network basically contains three gates to control the flow of information and cells to hold information. The Cell States carries the information from initial to later time steps without getting vanished.



LSTM Networks

Forget Gate:

- This gate decides what information should be carried out forward or what information should be ignored.
- Information from previous hidden states and the current state information passes through the sigmoid function. Values that come out from sigmoid are always between 0 and 1.
- if the value is closer to 1 means information should proceed forward and if value closer to 0 means information should be ignored.

LSTM Networks

Input Gate:

- After deciding the relevant information, the information goes to the input gate, Input gate passes the relevant information, and this leads to updating the cell states. simply saving updating the weight.
- Input gate adds the new relevant information to the existing information by updating cell states.

LSTM Networks

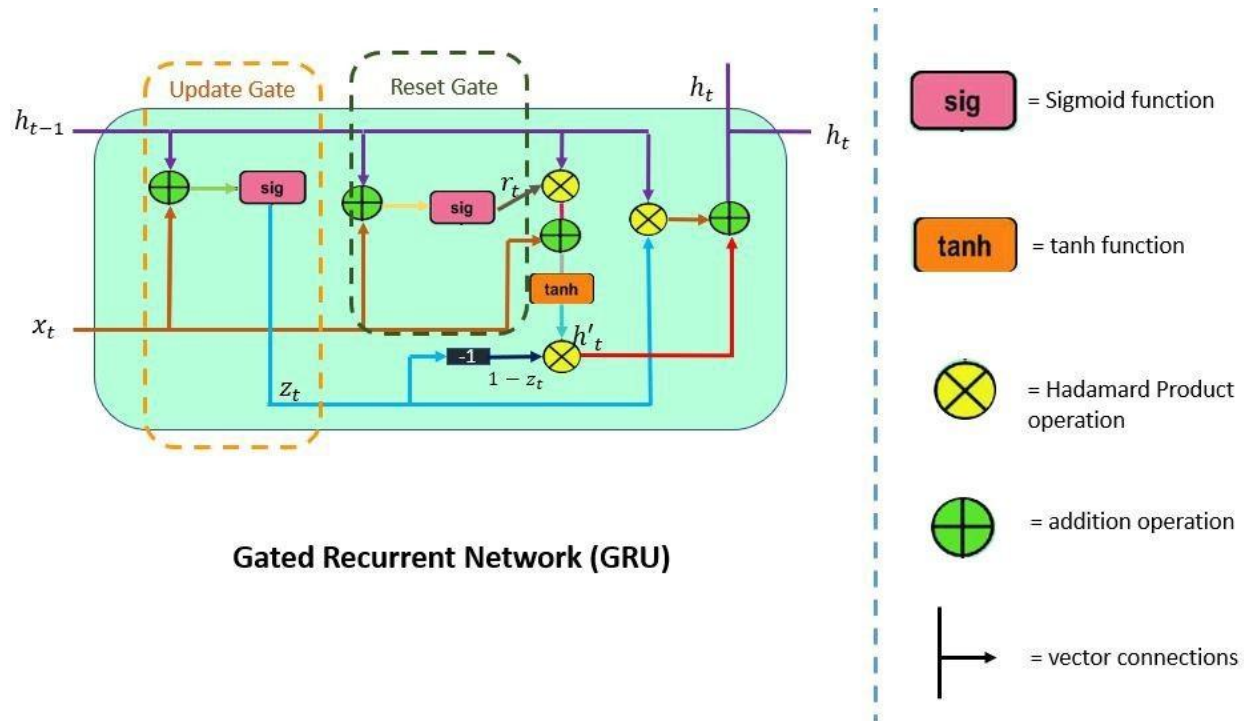
Output Gate:

- After the information is passed through the input gate, now the output gate comes into play.
- Output gate generates the next hidden states. and cell states are carried over the next time step.

GRU

GRU (Gated Recurrent Units) are similar to the LSTM networks. GRU is a kind of newer version of RNN. However, there are some differences between GRU and LSTM.

- GRU doesn't contain a cell state
- GRU uses its hidden states to transport information
- It Contains only 2 gates(Reset and Update Gate)
- GRU is faster than LSTM



Gated Recurrent Network (GRU)

GRU

1. Update Gate

- Update Gate is a combination of Forget Gate and Input Gate. Forget gate decides what information to ignore and what information to add in memory.

2. Reset Gate

- This Gate Resets the past information in order to get rid of gradient explosion. Reset Gate determines how much past information should be forgotten.

Other RNN Variants

GRU [*Learning phrase representations using rnn encoder-decoder for statistical machine translation*, Cho et al. 2014]

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r)$$

$$z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z)$$

$$\tilde{h}_t = \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

Summary

- RNNs allow a lot of flexibility in architecture design
- Vanilla RNNs are simple but don't work very well
- Common to use LSTM or GRU: their additive interactions improve gradient flow
- Backward flow of gradients in RNN can explode or vanish. Exploding is controlled with gradient clipping. Vanishing is controlled with additive interactions (LSTM)



UE23EC343BC1 DEEP LEARNING

Dr. Rashmi N Ugarakhod

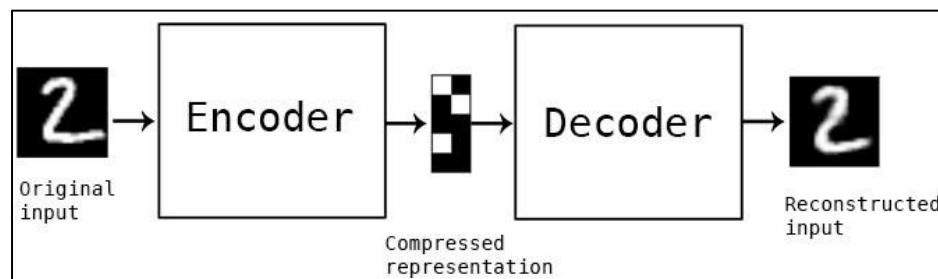
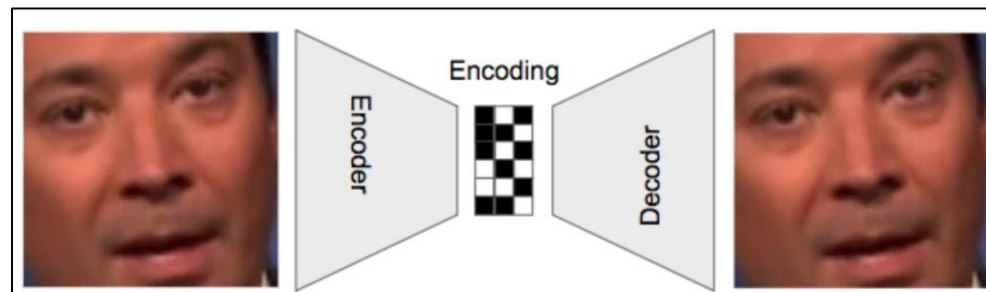
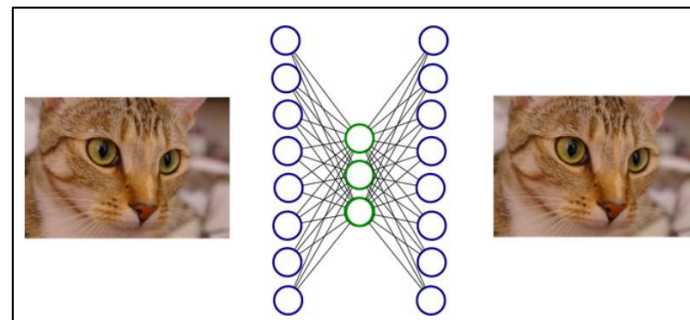
Department of Electronics and Communication Engineering

rashmin@pes.edu

Autoencoders

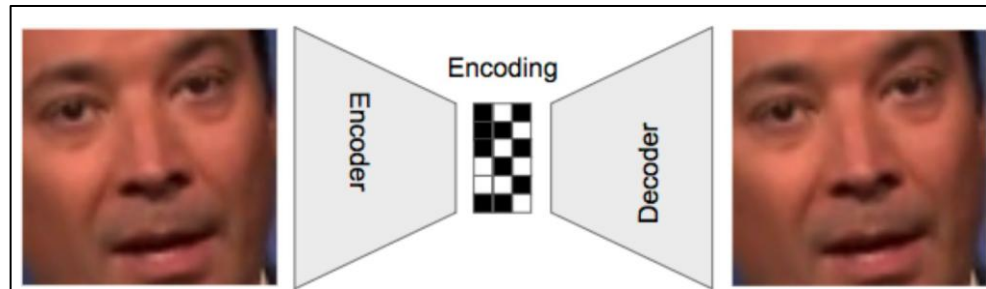
What is an Autoencoder?

- A neural network trained using unsupervised learning
 - Trained to copy its input to its output
 - Learns an embedding



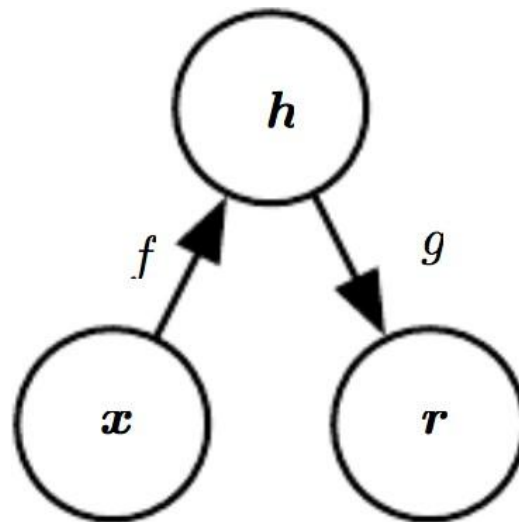
Embedding is a point on a manifold

- An embedding is a low-dimensional vector
 - With fewer dimensions than the ambient space of which the manifold is a low-dimensional subset
- Embedding Algorithm
 - Maps any point in ambient space x to its embedding h
 - Embeddings of related inputs form a manifold



General structure of an autoencoder

- Maps an input x to an output r (called reconstruction) through an internal representation code h
 - It has a hidden layer h that describes a code used to represent the input
- The network has two parts
 - The encoder function $h=f(x)$
 - A decoder that produces a reconstruction $r=g(h)$



What does an Autoencoder Learn?

- Learning $g(f(\mathbf{x}))=\mathbf{x}$ everywhere is not useful
- Autoencoders are designed to be unable to copy perfectly
 - Restricted to copy only approximately
- Autoencoders learn useful properties of the data
 - Being forced to prioritize which aspects of input should be copied
- Can learn stochastic mappings
 - Go beyond deterministic functions to mappings $p_{\text{encoder}}(\mathbf{h}|\mathbf{x})$ and $p_{\text{decoder}}(\mathbf{x}|\mathbf{h})$

Autoencoder History

- Part of neural network landscape for decades
 - Used for dimensionality reduction and feature learning
- Theoretical connection to latent variable models
 - Have brought them into forefront of generative models
 - Variational Autoencoders

Two Autoencoder Training Methods

1. Autoencoder is a feed-forward non-recurrent neural net
 - With an input layer, an output layer and one or more hidden layers
 - Can be trained using the same techniques
 - Compute gradients using back-propagation
 - Followed by minibatch gradient descent
2. Unlike feedforward networks, can also be trained using *Recirculation*
 - Compare activations on the input to activations of the reconstructed input
 - More biologically plausible than back-prop but rarely used in ML

1. Undercomplete Autoencoder

- Copying input to output sounds useless
- But we have no interest in decoder output
- We hope h takes on useful properties
- Undercomplete autoencoder
 - Constrain h to have lower dimension than x
 - Force it to capture most salient features of training data

Autoencoder with linear decoder +MSE is PCA

- Learning process is that of minimizing a loss function

$$L(\mathbf{x}, g(f(\mathbf{x})))$$

- where L is a loss function penalizing $g(f(\mathbf{x}))$ for being dissimilar from $\mathbf{x}$
 - such as L^2 norm of difference: mean squared error
- When the decoder g is linear and L is the mean squared error, an undercomplete autoencoder learns to span the same subspace as PCA
 - In this case the autoencoder trained to perform the copying task has learned the principal subspace of the training data as a side-effect
- Autoencoders with nonlinear f and g can learn more powerful nonlinear generalizations of PCA
 - But high capacity is not desirable

Encoder/Decoder Capacity

- If encoder f and decoder g are allowed too much capacity
 - autoencoder can learn to perform the copying task without learning any useful information about distribution of data
- Autoencoder with a one-dimensional code and a very powerful nonlinear encoder can learn to map $\mathbf{x}^{(i)}$ to code i .
 - The decoder can learn to map these integer indices back to the values of specific training examples
- Autoencoder trained for copying task fails to learn anything useful if f/g capacity is too great

Cases when Autoencoder Learning Fails

- Where autoencoders fail to learn anything useful:
 1. Capacity of encoder/decoder f/g is too high
 - Capacity controlled by depth
 2. Hidden code h has dimension equal to input x
 3. *Overcomplete* case: where hidden code h has dimension greater than input x
 - Even a linear encoder/decoder can learn to copy input to output without learning anything useful about data distribution

Right Autoencoder Design: Use regularization

- Ideally, choose code size (dimension of $\mathbf{h}$) small and capacity of encoder f and decoder g based on complexity of distribution modeled
- *Regularized autoencoders* provide the ability to do so
 - Rather than limiting model capacity by keeping encoder/decoder shallow and code size small
 - They use a loss function that encourages the model to have properties other than copy its input to output

2. Regularized Autoencoder Properties

- Regularized AEs have properties beyond copying input to output:
 - Sparsity of representation
 - Smallness of the derivative of the representation
 - Robustness to noise
 - Robustness to missing inputs
- Regularized autoencoder can be nonlinear and overcomplete
 - But still learn something useful about the data distribution even if model capacity is great enough to learn trivial identity function

Generative Models Viewed as Autoencoders

- Beyond regularized autoencoders
- Generative models with latent variables and an inference procedure (for computing latent representations given input) can be viewed as a particular form of autoencoder
- Generative modeling approaches which emphasize connection with autoencoders are descendants of Helmholtz machine:
 1. Variational autoencoder
 2. Generative stochastic networks

Sparse Autoencoder Loss Function

- A sparse autoencoder is an autoencoder whose
 - Training criterion includes a sparsity penalty $\Omega(\mathbf{h})$ on the code layer $\mathbf{h}$ in addition to the reconstruction error:
$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h})$$
 - where $g(\mathbf{h})$ is the decoder output and typically we have $\mathbf{h} = f(\mathbf{x})$
- Sparse encoders are typically used to learn features for another task such as classification
- An autoencoder that has been trained to be sparse must respond to unique statistical features of the dataset rather than simply perform the copying task
 - Thus sparsity penalty can yield a model that has learned useful features as a byproduct

Sparse Encoder doesn't have Bayesian Interpretation

- Penalty term $\Omega(\mathbf{h})$ is a regularizer term added to a feedforward network whose
 - Primary task: copy input to output (with *Unsupervised* learning objective)
 - Also perform some supervised task (with *Supervised* learning objective) that depends on the sparse features
- In supervised learning regularization term corresponds to prior probabilities over model parameters
 - Regularized MLE corresponds to maximizing $p(\boldsymbol{\theta}|\mathbf{x})$, which is equivalent to maximizing $\log p(\mathbf{x}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta})$
 - First term is data log-likelihood and second term is log-prior over parameters
 - Regularizer depends on data and thus is not a prior
 - Instead, regularization terms express a preference over functions

Generative Model view of Sparse Autoencoder

- Rather than thinking of sparsity penalty as a regularizer for copying task, think of sparse autoencoder as approximating ML training of a generative model that has latent variables
- Suppose model has visible/latent variables $\mathbf{x}$ and $\mathbf{h}$
- Explicit joint distribution is $p_{\text{model}}(\mathbf{x}, \mathbf{h}) = p_{\text{model}}(\mathbf{h}) p_{\text{model}}(\mathbf{x}|\mathbf{h})$
 - where $p_{\text{model}}(\mathbf{h})$ is model's prior distribution over latent variables
 - Different from $p(\theta)$ being distribution of parameters
- The log-likelihood can be decomposed as $\log p_{\text{model}}(\mathbf{x}, \mathbf{h}) = \log \sum_{\mathbf{h}} p_{\text{model}}(\mathbf{h}, \mathbf{x})$
- Autoencoder approximates the sum with a point estimate for just one highly likely value of $\mathbf{h}$, the output of a parametric encoder
 - With a chosen $\mathbf{h}$ we are maximizing $\log p_{\text{model}}(\mathbf{x}, \mathbf{h}) = \log p_{\text{model}}(\mathbf{h}) + \log p_{\text{model}}(\mathbf{x}|\mathbf{h})$

Sparsity-inducing Priors

- The $\log p_{\text{model}}(\mathbf{h})$ term can be sparsity-inducing. For example the Laplace prior

$$p_{\text{model}}(h_i) = \frac{\lambda}{2} e^{-\lambda |h_i|}$$

- corresponds to an absolute value sparsity penalty
- Expressing the log-prior as an absolute value penalty

$$-\log p_{\text{model}}(\mathbf{h}) = \sum_i (\lambda |h_i| - \log \frac{\lambda}{2}) = \Omega(\mathbf{h}) + \text{const}$$

- where the constant term depends only on λ and not on $\mathbf{h}$
- We treat λ as a hyperparameter and discard the constant term, since it does not affect parameter learning

Denoising Autoencoders (DAE)

- Rather than adding a penalty Ω to the cost function, we can obtain an autoencoder that learns something useful
 - By changing the reconstruction error term of the cost function
- Traditional autoencoders minimize $L(\mathbf{x}, g(f(\mathbf{x})))$
 - where L is a loss function penalizing $g(f(\mathbf{x}))$ for being dissimilar from $\mathbf{x}$, such as L^2 norm of difference: mean squared error
- A DAE minimizes $L(\mathbf{x}, g(f(\tilde{\mathbf{x}})))$
 - Where $\tilde{\mathbf{x}}$ is a copy of $\mathbf{x}$ that has been corrupted by some form of noise
 - The autoencoder must undo this corruption rather than simply copying their input
- Denoising training forces f and g to implicitly learn the structure of $p_{\text{data}}(\mathbf{x})$
- Another example of how useful properties can emerge as a by-product of minimizing reconstruction error

Regularizing by Penalizing Derivatives

- Another strategy for regularizing an autoencoder
- Use penalty as in sparse autoencoders

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h}, \mathbf{x})$$

- But with a different form of Ω

$$\Omega(\mathbf{h}, \mathbf{x}) = \lambda \sum_i \|\nabla_{\mathbf{x}} h_i\|^2$$

- Forces the model to learn a function that does not change much when $\mathbf{x}$ changes slightly
- Called a *Contractive Auto Encoder* (CAE)
- This model has theoretical connections to
 - Denoising autoencoders
 - Manifold learning
 - Probabilistic modeling

3. Representational Power, Layer Size and Depth

- Autoencoders are often trained with single layer
- However using deep encoder offers many advantages
 - Recall: Although universal approximation theorem states that a single layer is sufficient, there are disadvantages:
 1. no of units needed may be too large
 2. may not generalize well
- Common strategy: greedily pretrain a stack of shallow autoencoders

4. Stochastic Encoders and Decoders

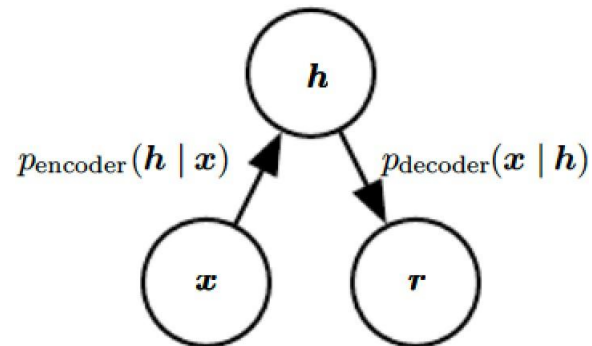
- General strategy for designing the output units and loss function of a feedforward network is to
 - Define the output distribution $p(\mathbf{y}|\mathbf{x})$
 - Minimize the negative log-likelihood $-\log p(\mathbf{y}|\mathbf{x})$
 - In this setting $\mathbf{y}$ is a vector of targets such as class labels
- In an autoencoder $\mathbf{x}$ is the target as well as the input

Loss function for Stochastic Decoder

- Given a hidden code $\mathbf{h}$, we may think of the decoder as providing a conditional distribution $p_{\text{decoder}}(\mathbf{x}|\mathbf{h})$
- We train the autoencoder by minimizing $-\log p_{\text{decoder}}(\mathbf{x}|\mathbf{h})$
- The exact form of this loss function will change depending on the form of $p_{\text{decoder}}(\mathbf{x}|\mathbf{h})$
- As with feedforward networks we use linear output units to parameterize the mean of the Gaussian distribution if $\mathbf{x}$ is real
 - In this case negative log-likelihood is the mean-squared error
- With binary $\mathbf{x}$ correspond to a Bernoulli with parameters given by a sigmoid
- Discrete $\mathbf{x}$ values correspond to a softmax
- The output variables are treated as being conditionally independent given $\mathbf{h}$

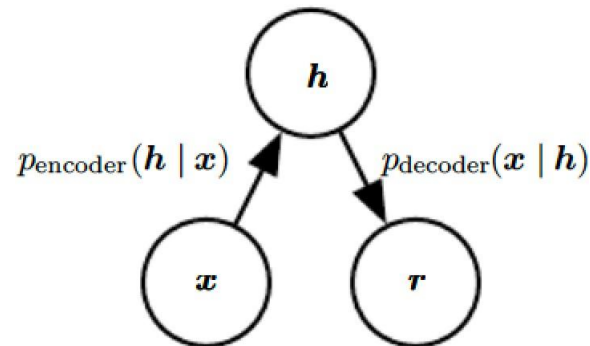
Stochastic encoder

- We can also generalize the notion of an encoding function $f(\mathbf{x})$ to an encoding distribution $p_{\text{encoder}}(\mathbf{h}|\mathbf{x})$



Structure of stochastic autoencoder

- Both the encoder and decoder are not simple functions but involve a distribution
- The output is sampled from a distribution $p_{\text{encoder}}(\mathbf{h}|\mathbf{x})$ for the encoder and $p_{\text{decoder}}(\mathbf{x}|\mathbf{h})$ for the decoder



Relationship to joint distribution

- Any latent variable model $p_{\text{model}}(\mathbf{h}|\mathbf{x})$ defines a stochastic encoder $p_{\text{encoder}}(\mathbf{h}|\mathbf{x})=p_{\text{model}}(\mathbf{h}|\mathbf{x})$
- And a stochastic decoder $p_{\text{decoder}}(\mathbf{x}|\mathbf{h})=p_{\text{model}}(\mathbf{x}|\mathbf{h})$
- In general the encoder and decoder distributions are not conditional distributions compatible with a unique joint distribution $p_{\text{model}}(\mathbf{x},\mathbf{h})$
- Training the autoencoder as a denoising autoencoder will tend to make them compatible asymptotically
 - With enough capacity and examples



UE23EC343BC1 DEEP LEARNING

Dr. Rashmi N Ugarakhod

Department of Electronics and Communication Engineering

rashmin@pes.edu

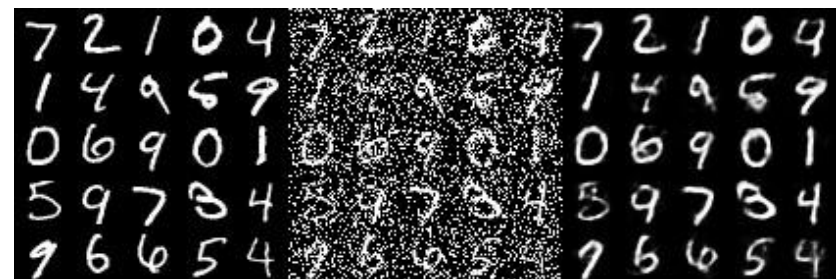
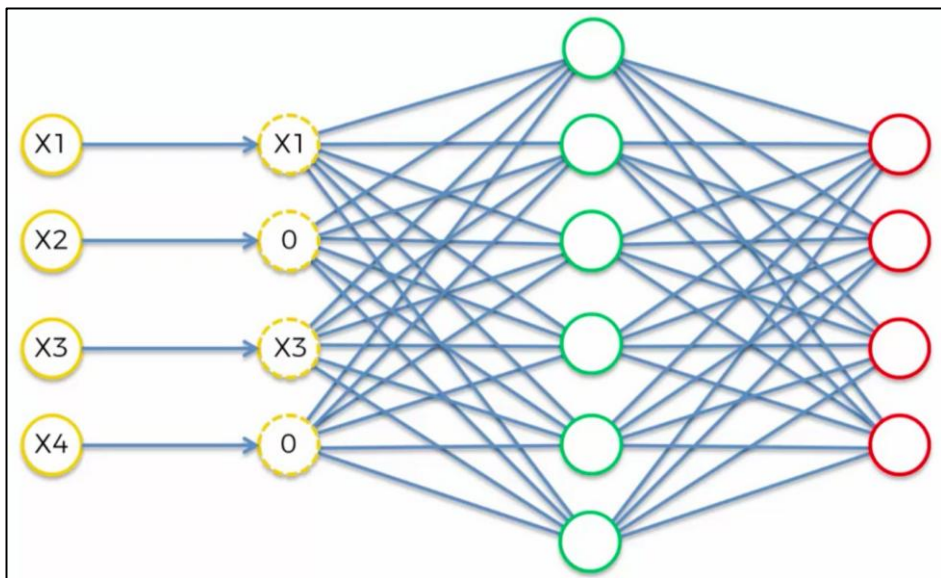
Denoising Autoencoders

Denoising Autoencoders

- An autoencoder that receives a corrupted data point as input and is trained to predict the original, uncorrupted data point as its output
 - Traditional autoencoders minimize $L(\mathbf{x}, g(f(\mathbf{x})))$
 - where L is a loss function penalizing $g(f(\mathbf{x}))$ for being dissimilar from $\mathbf{x}$, such as L^2 norm of difference: mean squared error
 - A DAE minimizes $L(\mathbf{x}, g(f(\tilde{\mathbf{x}})))$
 - Where $\tilde{\mathbf{x}}$ is a copy of $\mathbf{x}$ that is corrupted by some form of noise
 - The autoencoder must undo this corruption rather than simply copying their input

Example of Noise in a DAE

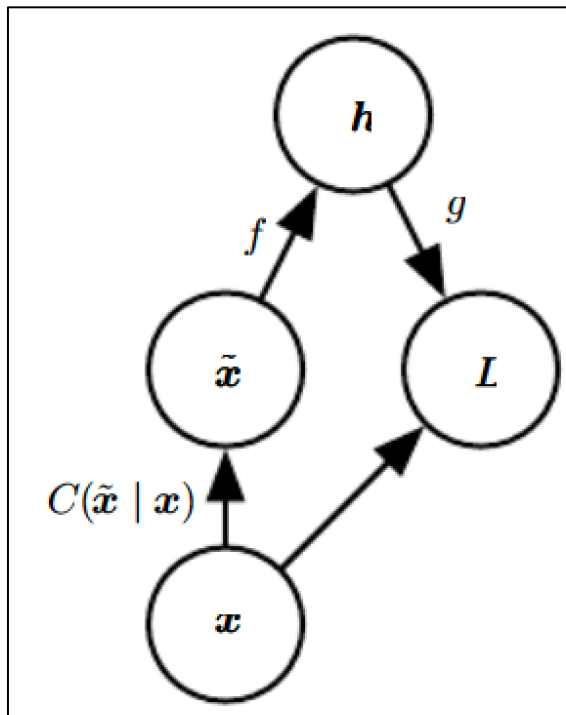
- An autoencoder with high capacity can end up learning an identity function (also called null function) where input=output
- A DAE can solve this problem by corrupting the data input
- How much noise to add?
 - Corrupt input nodes by setting 30-50% of random input nodes to zero



Original input, corrupted data, reconstructed data

DAE Training procedure

- Computational graph of cost function below
 - DAE trained to reconstruct clean data point $\mathbf{x}$ from the corrupted
Accomplished by minimizing loss $L = -\log p_{\text{encoder}}(\mathbf{x} | \mathbf{h} = f(\mathbf{x}))$



Corruption process, $C(\tilde{\mathbf{x}} | \mathbf{x})$ is a conditional distribution over corrupted samples $\tilde{\mathbf{x}}$ given the data sample $\mathbf{x}$

The autoencoder learns a reconstruction distribution

DAE Training procedure

Preconstruct($x|\tilde{x}$) estimated from training pairs $(x, \tilde{x})$ as follows:

1. **Sample a training sample x from the training data**
2. **Sample a corrupted version $\tilde{x}$ from $C(\tilde{x}|x)$**
3. **Use $(x, \tilde{x})$ as a training example for estimating the autoencoder distribution $p_{\text{reconstruct}}(x|\tilde{x}) = p_{\text{decoder}}(x|h)$ with h the output of encoder $f(\tilde{x})$ and p_{decoder} typically defined by a decoder $g(h)$**

Estimating the Score

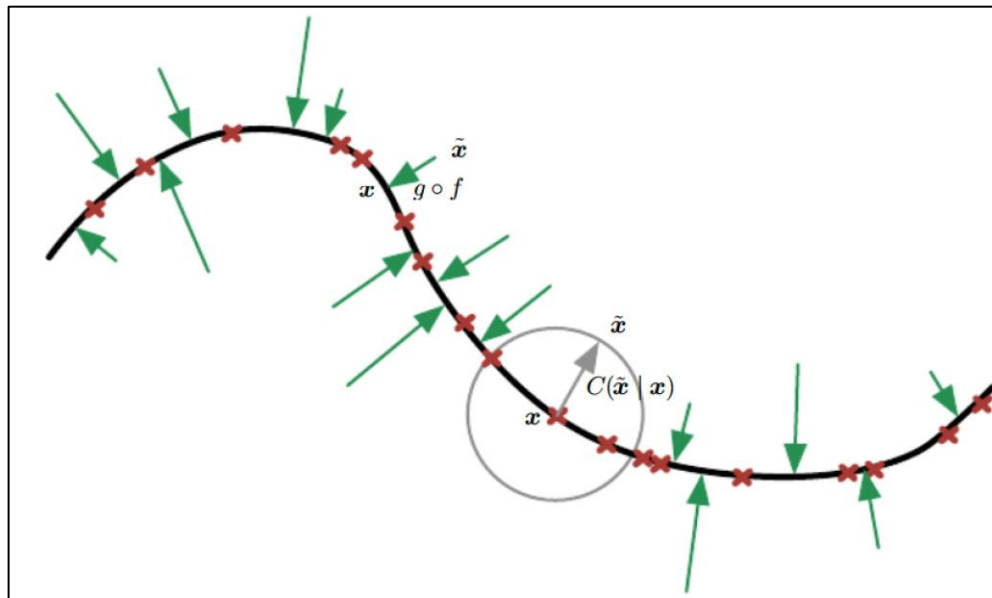
- An autoencoder can be based on encouraging the model to have the same score as the data distribution at every training point $\mathbf{x}$
 - The score is a particular gradient field is: $\nabla_{\mathbf{x}} \log p(\mathbf{x})$
 - Learning the *gradient field* of $\log p_{\text{data}}$ is one way to learn the structure of p_{data} itself
- Score Matching works by fitting the slope (score) of the model density to the slope of the true underlying density at the data points
- DAE, with conditionally Gaussian $p(\mathbf{x}|\mathbf{h})$, estimates this score as $(g(f(\mathbf{x}))-x)$

The DAE is trained to minimize $\|g(f(\tilde{x}) - x)\|^2$

- DAE estimates a vector fields as illustrated next

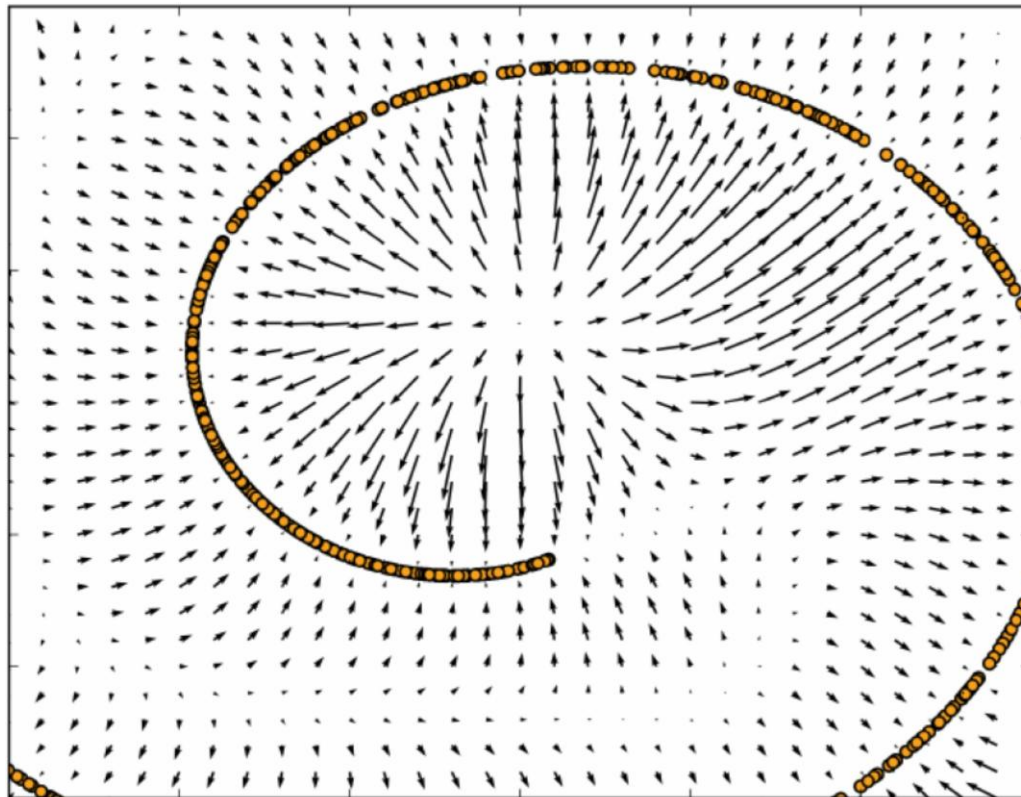
DAE learns a vector field

- Training examples $\mathbf{x}$ lie on a low-dimensional manifold
 - Training examples $\mathbf{x}$ are red crosses
- Gray circle is equiprobable corruptions
- The vector field $(g(f(\mathbf{x}) - \mathbf{x}))$, indicated by green arrows, estimates the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ which is the slope of the density of data



Vector field learnt by a DAE

- 1-D curved manifold near which the data concentrate
- Each arrow proportional to reconstruction minus input vector of DAE and points towards higher probability
- Where probability is maximum arrows shrink





UE23EC343BC1 DEEP LEARNING

Dr. Rashmi N Ugarakhod

Department of Electronics and Communication Engineering

rashmin@pes.edu

Applications

Applications

- Autoencoders have been successfully applied to dimensionality reduction and information retrieval tasks.
- Dimensionality reduction was one of the first applications of representation learning and deep learning. It was one of the early motivations for studying autoencoders.
- For example, [Hinton and Salakhutdinov \(2006\)](#) trained a stack of RBMs and then used their weights to initialize a deep autoencoder with gradually smaller hidden layers, culminating in a bottleneck of 30 units.
- The resulting code yielded less reconstruction error than PCA into 30 dimensions and the learned representation was qualitatively easier to interpret and relate to the underlying categories, with these categories manifesting as well-separated clusters.

Applications

- Lower-dimensional representations can improve performance on many tasks, such as classification. Models of smaller spaces consume less memory and runtime.
- Many forms of dimensionality reduction place semantically related examples near each other, as observed by Salakhutdinov and Hinton (2007b) and Torralba *et al.* (2008).
- The hints provided by the mapping to the lower-dimensional space aid generalization.

Applications

- One task that benefits even more than usual from dimensionality reduction is **information retrieval**, the task of finding entries in a database that resemble a query entry.
- This task derives the usual benefits from dimensionality reduction that other tasks do, but also derives the additional benefit that search can become extremely efficient in certain kinds of low dimensional spaces.
- Specifically, if we train the dimensionality reduction algorithm to produce a code that is lowdimensional and *binary*, then we can store all database entries in a hash table mapping binary code vectors to entries.

Applications

- This hash table allows us to perform information retrieval by returning all database entries that have the same binary code as the query. We can also search over slightly less similar entries very efficiently, just by flipping individual bits from the encoding of the query.
- This approach to information retrieval via dimensionality reduction and binarization is called **semantic hashing** (Salakhutdinov and Hinton, 2007b, 2009b), and has been applied to both textual input (Salakhutdinov and Hinton, 2007b, 2009b) and images (Torralba *et al.*, 2008; Weiss *et al.*, 2008; Krizhevsky and Hinton, 2011).

- To produce binary codes for semantic hashing, one typically use encoding function with sigmoid on the final layer.
- The sigmoid units must be trained to be saturated to nearly 0 or nearly 1 for all input values.
- One trick that can accomplish this is simply to inject additive noise just before the sigmoid nonlinearity during training.
- The magnitude of the noise should increase over time. To fight that noise and preserve as much information as possible, the network must increase the magnitude of the inputs to the sigmoid function, until saturation occurs.
- The idea of learning a hashing function has been further explored in several directions, including the idea of training the representations so as to optimize a loss more directly linked to the task of finding nearby examples in the hash table (Norouzi and Fleet, 2011).